

[Intro](#)

[Modules](#)

[Netmiko Library](#)

[Scripting with Python](#)

[Python Interactive Shell](#)

[Editors and IDEs](#)

[Python Data Types](#)

[String Data Type](#)

[Integer Data Type](#)

[Float Data Type \(# w/ decimal\)](#)

[Boolean Data Type \(true or false\)](#)

[type\(\) Function](#)

[Python Variables](#)

[List Analysis \[\]](#)

[Tuple Data Type \(, \)](#)

[Set Data Type { }](#)

[Dictionary](#)

[Nested Data Types](#)

[Navigating Complex Data Structures](#)

[Manipulating Strings](#)

[String Manipulation](#)

[String Splitting](#)

[Modifying Strings \(Slicing\)](#)

[String Concatenation \(+\)](#)

[Whitespace Stripping](#)

[Formatting and Templating](#)

[Escape Characters](#)

[Regular Expressions](#)

[String Methods](#)

[Mac Addresses](#)

[Describing Conditionals, Loops, and Operators](#)

[Whitespace and PEP 8](#)

[Conditionals](#)

[Operators](#)

[Loops](#)

[Loops with Lists, Dicts, and Ranges](#)

[Loops with Lists](#)

[Loops with Dictionaries](#)

[Loops with Ranges](#)

[Loops with Conditionals](#)

[Exploring Classes, Methods, Functions, Namespaces, and Scopes](#)

[Functions](#)

[Code Commenting](#)

[Code Commenting: Docstring](#)

[Namespaces and Scopes](#)

[Classes and Methods](#)

[Using Inheritance to Extend Functionality](#)

[Main Construct](#)

[Data Storage Options](#)

[Data Formats](#)

[Reading from a Simple Text and CSV File](#)

[Writing Data to a Simple Text and CSV File](#)

[Note](#)

[Reading Data from a JSON Text File](#)

[Writing Data to a JSON Text File](#)

[Data Access in Raw or Unstructured Format](#)

[Python Modules and Packages](#)

[Python Standard Library Modules](#)

[Reusable Code with Modules and Packages](#)

[Gathering and Validating User Input](#)

[Obtaining User Input](#)

[Note](#)

[Command-Line Arguments](#)

[Analyzing Exceptions and Error Management](#)

[Errors and Exceptions](#)

[Managing Exceptions](#)

[Assertions](#)

[Crash Course Partial Notes](#)

[Linux](#)

[Definitions](#)

The Programming for Network Engineers course

Intro

REST = REpresentational State Transfer
blueprint or architecture which uses HTTP
nomenclature usage example, 'a RESTful HTTP Request'
REST APIs

NETCONF is a protocol that is defined by the IETF to address some of the challenges with CLI scripting by providing a standards-based method to "install, manipulate, and delete the configuration of network devices." NETCONF typically uses SSH for secure transport and XML to exchange data. NETCONF provides remote procedure calls (RPCs) to operate network devices in a programmatic fashion. In order for NETCONF to be useful as a network-wide protocol, it must have a common data model. YANG is a modeling language defined in IETF RFC 6020. YANG is used by NETCONF to define the objects and data in requests and replies.

RESTCONF is a protocol that exposes a REST-like interface to network devices and uses HTTP for transport and JSON or XML to interface with network devices exposing data modeled in YANG. Both RESTCONF and NETCONF enable access to the same YANG modeled data, but they use different API bindings. Note that RESTCONF does not replace NETCONF. RESTCONF just provides another mechanism to expose the same data modeled via YANG (such as configuration or operational data). Programmers or designers may choose which API to leverage based on experience, skill set, and requirements.

Languages

Compiled vs Interpreted

- python can be both

Statically vs Dynamically typed

- python is dynamically

Structured/Procedural vs Object Oriented (OO)

- python is structured but can also be used as an OO language

- in python, everything is an object

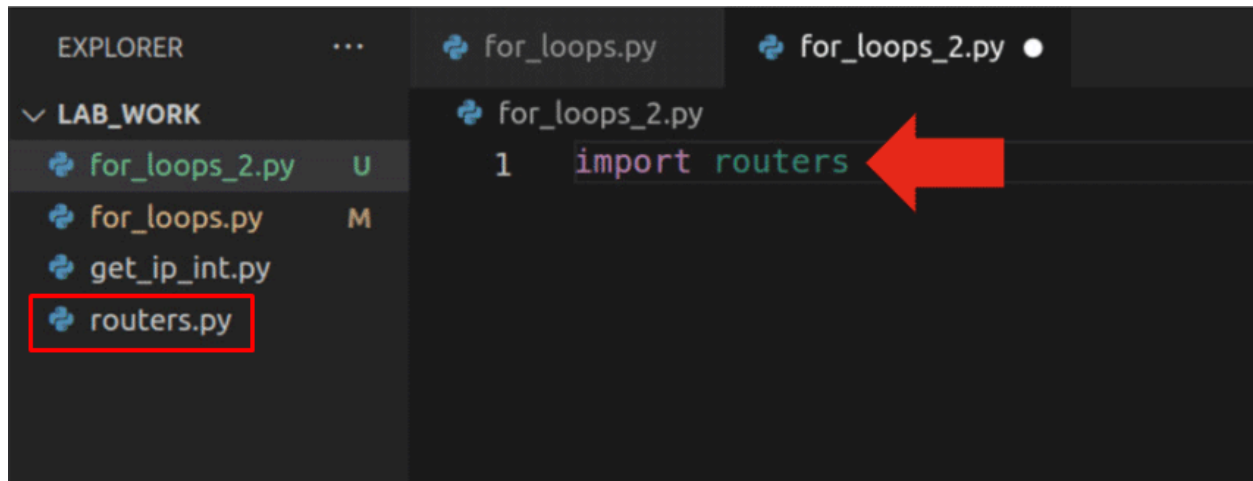
Imperative vs. Declarative

Imperative versus Declarative. In an imperative language, the programmer explicitly tells the system how to accomplish something. A declarative language attempts to allow the programmer to specify what is to be done, and the language figures out how to do it. In general, programs that are written using a declarative language are more concise than programs that are written in an imperative language because the programmer does not have to code every action to be taken.

Python is imperative in nature, but it is possible to create a declarative system using Python.

Modules

modules may be pre-created python modules or they may be a reference to another script:



Netmiko Library

Netmiko has a library for SSH and passing commands to network devices for example, SSH parameters are handled by the connect handler:

```
net_connect = netmiko.ConnectHandler(  
    ip = ip,  
    device_type = device_type,  
    username = username,  
    password = password,  
    port = port  
)
```

and the command is sent as part of the connect handler :

```
show_version = net_connect.send_command('show version')  
print(show_version)
```

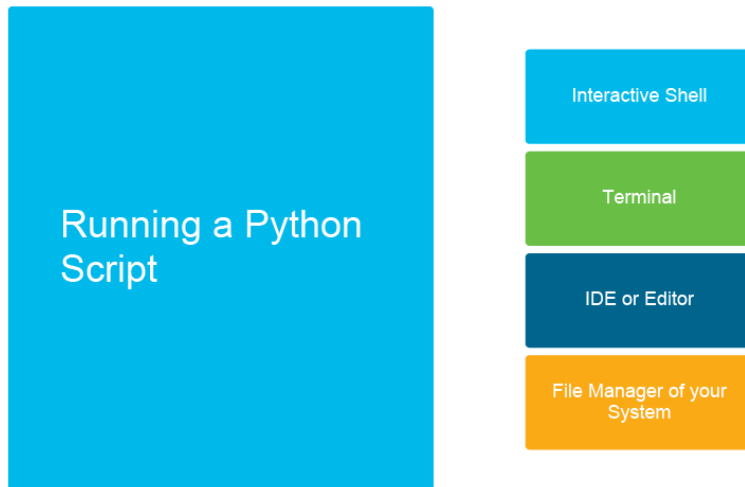
basic SSH connect script with send command:

```
import netmiko
```

```
ip = '10.254.0.1'  
username = 'cisco'  
password = 'cisco'  
device_type= 'cisco_ios'  
port = '22'
```

```
net_connect = netmiko.ConnectHandler(ip=ip, device_type=device_type, username=username,  
password=password, port=port)  
show_arp = net_connect.send_command('show arp')  
print(show_arp)
```

Scripting with Python



Python is an interpreted language

Compiles code into byte code

Line by line execution

Easy to get started

- **Interactive Shell:** The Python interactive shell allows you to execute commands line by line.

Interactive Shell

```
C:\Users\chips\Documents>python
Python 3.9.1 (tags/v3.9.1:1e5d33e, Dec 7 2020, 17:08:21) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

- **Terminal:** The Windows and Linux terminals can execute Python code.

```
C:\Users\chips\OneDrive\Documents>python HelloPRNE.py
What is your name?Bill
Bill, welcome to the PRNE class!

C:\Users\chips\OneDrive\Documents>
```

Terminal

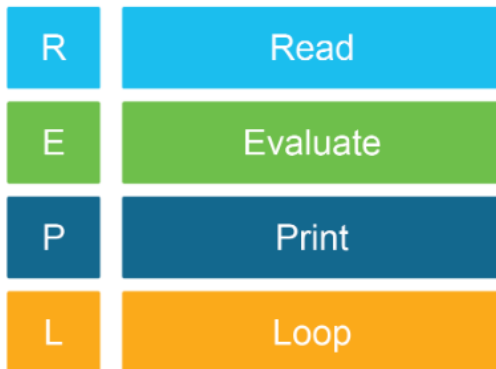
- **IDE/Editor:** The integrated execution in your IDE or Editor, such as VS Code, can execute scripts.
- **File manager of your system:** The system's file manager allows you to execute a script by double-clicking on the script's icon. This method is not typically used during development, but it may be used when the script is released to production.

Python Interactive Shell

```
Command Prompt - Python
Microsoft Windows [Version 10.0.19042.867]
(c) 2020 Microsoft Corporation. All rights reserved.

C:\Users\chips>Python
Python 3.9.1 (tags/v3.9.1:1e5d33e, Dec  7 2020, 17:08:21) [MSC v.1927 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

REPL



Python is an interpreted language, which means it can execute code line by line. Executing code line by line is useful for learning Python and testing code concepts without having to compile and run a file.

When using the interpreter, the commands are executed and the results are displayed in the Python shell. The shell is also known as REPL. REPL (Read, Evaluate, Print, Loop) represents the process that the interpreter uses.

- R: Reads user input
- E: Evaluates the code
- P: Prints (displays) the output to the user
- L: Loops back to step read

The REPL process involves the following steps. Once you enter the interactive shell, you are presented with three chevrons (>>>), which means the shell is waiting for instructions. You then type a command and press <enter>. The computer will evaluate the command; if there is anything to display to the user, the computer will show it on the screen.

To check python version:

```
python --version
```

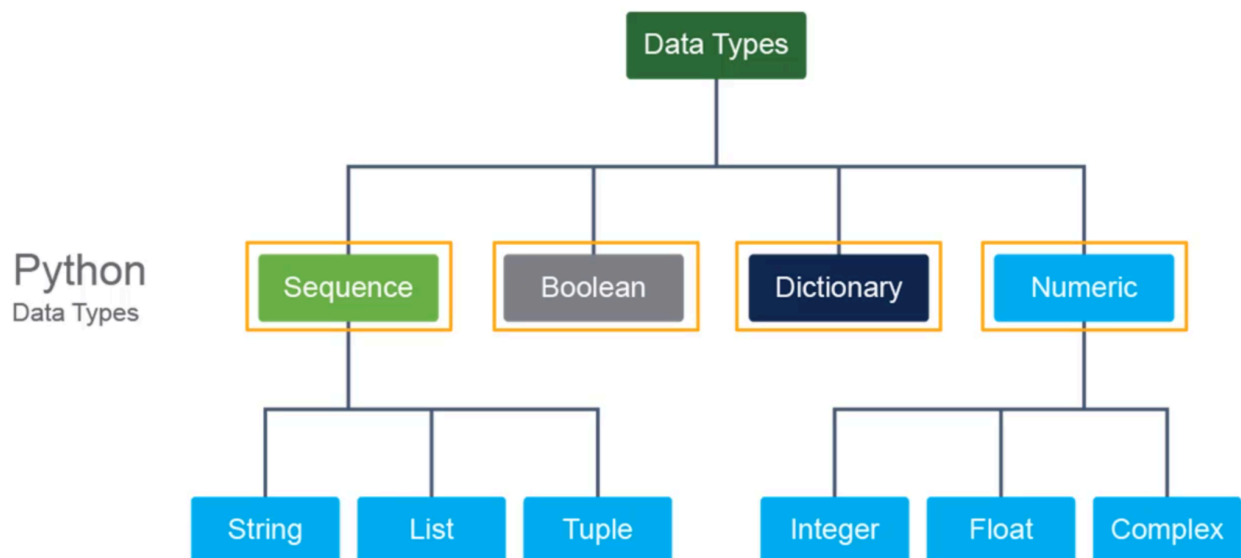
Editors and IDEs



Code Editor e.g. Notepad++, Visual Studio Code, Sublime Text, Vim

Integrated Development Environment (IDE) e.g. PyCharm, IDLE, Visual Studio, Spyder

Python Data Types



complex data type is used for imaginary numbers

String Data Type

Strings

Strings are a series of characters. The characters in a string can be alphanumeric including many special characters such as punctuation.

Quotes

Strings can be denoted by either single (') or double (") quotation marks. For example, 'this is my string'. However, you must be consistent. If you need to have one type of quote as actual data within a string, you can use the other type of quote to delimit the string. If you wish to extend a single string across multiple lines, you can use triple quotes at the beginning and ending of the string.

Single Quotes

'Allows Embedded "Double" Quotes'

Double Quotes

"Allows Embedded 'Single' Quotes"

Triple Quotes

"""Single Quotes or Double Quotes can be Used to Span Strings Over Multiple Lines"""

Error

'This is Bill's Script'

```
1 print('Router1')
2 print("192.168.1.1")
3 print("""Triple Double Quotes""")
4 print('''Single Triple Quotes''')
5 print("Du'An Lightfoot")
6 print('The Engineer said: "I love Cisco!"')
7
8
```

```
TERMINAL
duan@cisco:~/Documents/prne$
Router1
192.168.1.1
Triple Double Quotes
Single Triple Quotes
Du'An Lightfoot
The Engineer said: "I love Cisco!"
```

Python uses the PEP8 Style guide for using string quotes

Integer Data Type

An Integer is a whole number with no decimal places.

```
>>> 25
>>> -25
>>> 25000000
>>> 25_000_000
```

Integers

```
>>> 25,000,000
(25, 0, 0)
<class 'tuple'>
```

Commas cannot be used in integers.

```
>>> int(4.55)
4
```

Converting to Integer

```

1  print(3)
2  print(-3)
3  print(200378)
4  print(-200_378)
5  print(25_000_000)
6  print(type(-25_000_000))
7
8

```

TERMINAL

```

duan@cisco:/bin/python3.9 /home/
3
-3
200378
-200378
25000000
<class 'int'>

```

```

1  print("23")
2  print(type("23"))
3  print(int("23"))
4  print(type(int("23")))
5
6
7
8

```

TERMINAL

```

duan@cisco:/bin/python3.9 /ho
23
<class 'str'>
23
<class 'int'>

```

In general there are two types of numbers with Python:

commas can't be used, but you can use underscores, which are disregarded

Integers. Integers are whole numbers, which can be very large, and which can also be negative. You can also use other numerical bases with integers, such as binary, octal, and hexadecimal.

you can convert strings to integers and floats to integers

You also can convert numbers with decimals, such as 4.55, or strings, such as "4.55", to an integer. This is possible because Python data types are not strongly typed. There is a built-in function called `int()` that converts non-int values to ints. The example shows how to use `int()` and the results:

```
>>> int(4.55)
4
>>> int("4.55")
4
```

Float Data Type (# w/ decimal)

Floats. Python provides the floating-point data type for precise numerical values such as 1.23. Floats can also use exponential notation. For example, 1.0e6 which is 1,000,000.

A floating point is a whole number with a decimal place.

```
>>> 1000000.0
>>> -4.5
>>> 4.5
>>> 1e6
1000000.0
```

Floating Point

```
>>> .123456789123456789
0.123456789123456789
```

17 Digits

```
>>> float(4)
4.0
>>> float("4.5")
4.5
```

Converting to Floating Point

only 17 digits displayed before converted to exponential form

```
>>> x = 1999999999999999999.99999
>>> print(x)
2e+19
```

you can convert numbers to floating-points:

```
>>> float(4)
4.0
>>> float("4.55")
4.55
```

example:

```
ip_base = '10.254.'
```

```
ip_end = 0.1
```

```
while ip_end <= 0.3:
    ip = ip_base + str(ip_end)
    ip_int_brief = get_ip_int_br(ip)
    print(ip_int_brief)
    print('IP int from ' + ip)
    print('_' * 80)
    ip_end += .1
```

```
student@workstation:~/lab_work$ python floats.py
Interface      IP-Address      OK? Method Status Protocol
GigabitEthernet1  unassigned      YES manual down   down
GigabitEthernet2  10.12.0.1       YES NVRAM  up     up
GigabitEthernet3  10.13.0.1       YES NVRAM  up     up
GigabitEthernet4  10.254.0.1      YES NVRAM  up     up
IP int from 10.254.0.1

Interface      IP-Address      OK? Method Status Protocol
GigabitEthernet1  10.12.0.2       YES NVRAM  up     up
GigabitEthernet2  unassigned      YES NVRAM  down   down
GigabitEthernet3  10.23.0.2       YES NVRAM  up     up
GigabitEthernet4  10.254.0.2      YES NVRAM  up     up
IP int from 10.254.0.2

student@workstation:~/lab_work$
```

3rd router 10.254.0.3 is not reached

The beginning value of `ip_end` is 0.1. Adding .1 to that twice should equal 0.3. When you do **math in Python, fractions are converted to binary fractions, the calculation occurs, then they are converted back into decimal fractions.** Sometimes the conversion

to and from binary fractions results in a negligible difference between what the result is and what you expected it to be. In this example, your third time through the loop, the value of `ip_end` is actually 0.30000000000000004.

This may not be a common usage of floats, but it is important to keep an open mind when writing code; your code must be designed to deal with a specific task or issue.

to fix:

round() function to make `ip_end` hold the value you expect. In your script, insert the below line of code as the last line of the while loop. Be sure to **match the indentation**

```
while ip_end <= 0.3:
    ip = ip_base + str(ip_end)
    ip_int_brief = get_ip_int_br(ip)
    print(ip_int_brief)
    print('IP int from ' + ip)
    print('_' * 80)
    ip_end += .1
    ip_end = round(ip_end, 2)
```

Boolean Data Type (true or false)

The Boolean data type can have one of two values, True or False. Non-Boolean objects such as integers, floats, and strings can also be evaluated in a Boolean context and determined to be True or False. In the examples, you use the **bool()** function to show how an integer or string can be evaluated to be True or False. If the number is zero, it will always evaluate as False. Any number other than zero (5, 6.5, -4.5) will evaluate to True. With strings, a blank or empty string will evaluate as False. Otherwise, a string will evaluate to True. Evaluating a string to a Boolean value is an easy way to determine if a user entered a value when an input was requested by using the `bool()` function on the variable where the value was stored. If it evaluates to True, there was an input.

```
>>> bool(1)
True
>>> bool(0)
False
```

```
>>> bool("PRNE")
True
>>> bool("")
False
```

Boolean Values

example:

```
>>> username = input("Please enter a username:")
Please enter a username:bill
>>> bool(username)
True
```

type() Function

use the type() to verify what data type an object is (usually for troubleshooting purposes or to identify an object the API has returned)

type(object) – returns the data type of an object.

```
>>> type(25_000_000)
<class 'int'>
```

```
>>> type(5.5)
<class 'float'>
```

```
>>> type(True)
<class 'bool'>
```

Python Variables

variable names are case sensitive

Correct Assignment Example	• <code>x = 5</code> • <code>switch_os = "IOS-XE"</code>
Wrong Assignment Example	<code>x == 5</code>
Naming Convention	Lowercase, words should have "_" between them.
Variable Types	Global and Local
Dynamically Typed	Variable data types are not declared.

note `x == 5` would compare x to 5 to see if they're equal

variables cannot start with a number

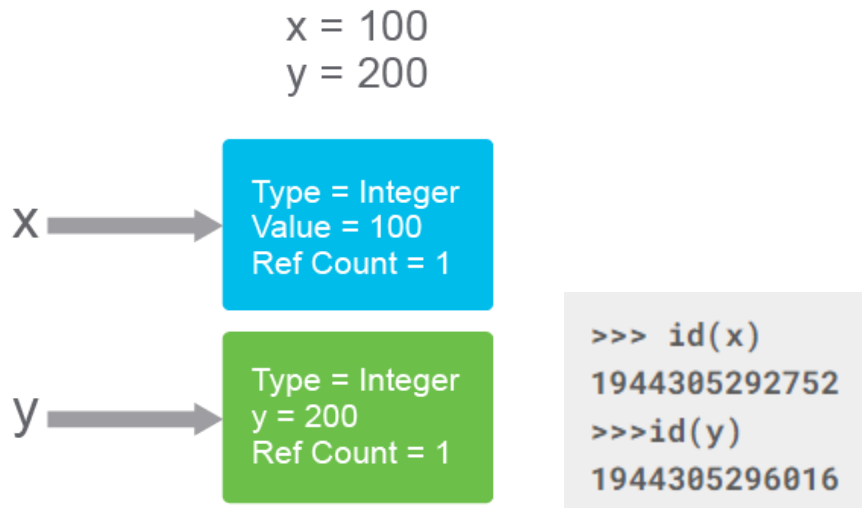
Variable names cannot start with a number.

```
>>> 1_name = 'cisco'
      File "<stdin>", line 1
        1_name = 'cisco'
        ^
SyntaxError: invalid token
```

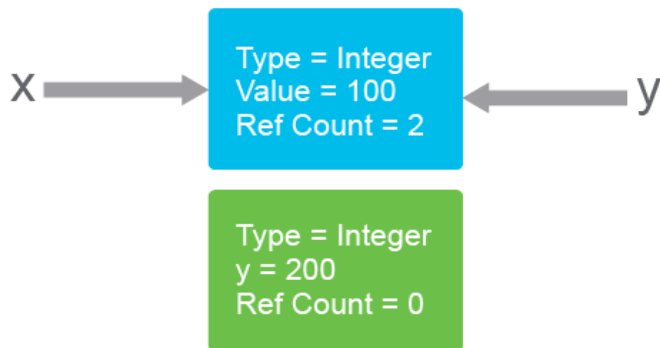
multiple word variables are separated with an underscore

```
>>> x = 1
>>> device_vendor = 'cisco'
>>> print(x, device_vendor)
1 cisco
```

each variable points to a reference in memory, which can be checked with the ID function



if both x and y = 100:



List Analysis []

Lists have several traits:

- Start and end with square brackets []
- Are ordered
- Contain any object
- Can be indexed (there can be duplicates)
- Are mutable
- Are dynamic

example:

```
>>> devices = ['router1', 'router2', 'switch1']
```

Verify:

```
>>> devices  
['router1', 'router2', 'switch1']
```

print by index:

```
>>> devices[0]  
'router1'
```

index:

```
>>> devices = ["router1", "router2", "switch1", 5]
```

-4 -3 -2 -1
0 1 2 3

to reference last item of list, use -1, 2nd to last item -2, etc.

Let's examine some methods that allow you to add, remove, and count objects in a list.

The List	>>> devices = ["router1", "router2", "router3"]
Add an object to the end of a list.	>>> devices.append("switch1") >>> devices ["router1", "router2", "router3", "switch1"]
The List	>>> devices = ["router1", "router2", "router3", "switch1"]
Add an object to a specific location in a list.	>>> devices.insert(3, "router4") >>> devices ["router1", "router2", "router3", "router4", "switch1"]

insert uses the index number for which position to insert the object

removal:

The List	<pre>>>> devices = ["router1", "router2", "router3", "router4", "switch1"]</pre>
Remove an object from the end of a list.	<pre>>>> devices.pop() 'switch1' >>> devices ['router1', 'router2', 'router3', 'router4']</pre>
The List	<pre>>>> devices = ["router1", "router2", "router3", "router4", "switch1"]</pre>
Remove an object from a specific index location in a list.	<pre>>>> devices.pop(3) 'router4' >>> devices ['router1', 'router2', 'router3', 'switch1']</pre>
The List	<pre>>>> devices = ["router1", "router2", "router3", "router4", "switch1"]</pre>
Remove an element from a list.	<pre>>>> devices.remove("switch1") >>> devices ['router1', 'router2', 'router3', 'router4', 'router1']</pre>

to Count number of times an element appears in list:

The List	<pre>>>> devices=["router1", "router2", "router3", "router4", "switch1", "router1"]</pre>
Count the number of times an object appears in a list.	<pre>>>> devices.count("router1") 2</pre>

to count the list:

```
>>> len(devices)
6
```

to get index of an element (this works for lists [] also)

Index Method

```
>>> os_type = ("IOS", "IOS-XE", "IOS-XR", "NX-OS")
>>> os_type.index("IOS-XR")
2
```

Returns the index for IOS-XR

Tuple Data Type (,)

immutable list = (,)

Parentheses Starts and Ends the Tuple

```
>>> os_type = ("IOS", "IOS-XE", "IOS-XR", "NX-OS")
```

```
>>> os_type = ("IOS", "IOS-XE", "IOS-XR", "NX-OS")
```

-4	-3	-2	-1
"IOS"	"IOS-XE"	"IOS-XR"	"NX-OS"
0	1	2	3

```
>>> os_type = ("IOS", "IOS-XE", "IOS-XR", "NX-OS")
```

```
>>> os_type[2]
```

```
'IOS-XR'
```

```
>>> os_type[-4]
```

```
'IOS'
```

to get index of an element:

Index Method

```
>>> os_type = ("IOS", "IOS-XE", "IOS-XR", "NX-OS")
```

```
>>> os_type.index("IOS-XR")
```

```
2
```

Returns the index
for IOS-XR

Set Data Type {}

uses curly brackets {}

contains any object

unordered and not indexed

no duplicates

are mutable

are dynamic

Sets are helpful for determining duplicates and removing duplicates

example: list of IP addresses or hostnames

example use: apply set to a list, to remove duplicates

basic use:

```
>>> os_type = {"IOS", "IOS-XE", "IOS-XR", "NX-OS"}
```

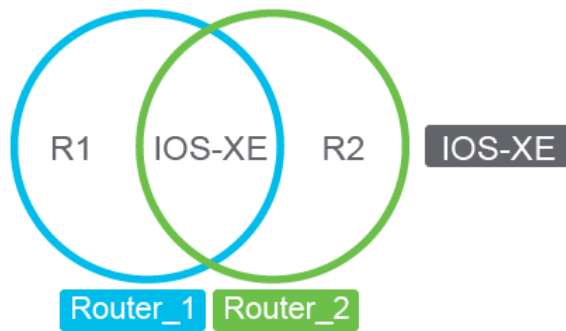
example use case is to convert a list to set to remove duplicates, then convert back to list. the example below removes the duplicate R1 from the list (still needs to be converted back to list)

Example 2

```
>>> hostnames = set(['R1', 'R2', 'R1', 'R3', 'R4'])
>>> hostnames
{'R3', 'R2', 'R1', 'R4'}
```

2 common set methods:

Union joins two sets and removes duplicates between the two



```
Union()
>>>Router_1.union(Router_2)
{R1, R2, IOS-XE}
```

Intersection finds duplicates between two sets:

```
Intersection()
>>>Router_1.intersection(Router_2)
{IOS-XE}
```

Dictionary

mutable key-valued pairs

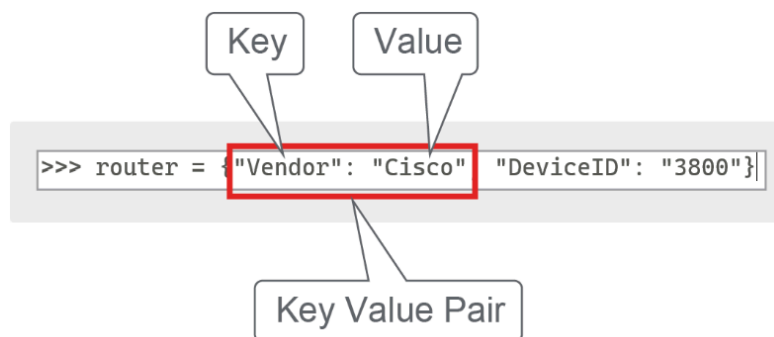
dictionaries use curly brackets {}

each key-value separated by commas

keys can be any immutable object

values can be any object

are dynamic



```
>>> router = {"Vendor": "Cisco", "DeviceID": "3800" }
```

Curly Brackets Start and End Your Dictionary

Building from an Empty Dictionary

```
>>> switch = {}
>>> switch['vendor']='Cisco'
>>> switch['DeviceModel']='Nexus 9000'
>>> switch['OS']='NX-OS'
>>> switch['Location']='RTP'
>>> switch
{'vendor': 'Cisco', 'DeviceModel': 'Nexus 9000', 'OS': 'NX-OS', 'Location': 'RTP'}
```

to retrieve the value of a key:

```
>>> router = {"vendor": "Cisco", "DeviceID": "3800", "Location": "San Jose"}
>>> router["vendor"]
'Cisco'
```

Retrieving Value of Key

```
>>> router["Location"]="Dallas"
>>> router
{'vendor': 'Cisco', 'DeviceID': '3800', 'Location': 'Dallas'}
```

Changing Key Value

ERROR:

```
>>> router["OS"]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'OS'
```

Error if Key Doesn't Exist

Methods you can use on dictionaries:

The example shows the `get()`, `items()`, `keys()` and `values()` methods.

- **.get():** Retrieves the **value of a given key**. Recall the earlier KeyError when attempting to access the value of a key not in the dictionary. The get() method will return a value of 'None' instead of an error.
- **.items():** Returns all the keys and values in a tuple form in a list
- **.keys():** Returns all the keys in a list format
- **.values():** Returns all the values in a list format

.get() method

```
>>> switch
{'vendor': 'Cisco', 'DeviceModel': 'Nexus 9000', 'OS': 'NX-OS', 'Location': 'RTP'}
>>> switch.get('OS')
'NX-OS'
```

.items() method

```
>>> switch.items()
dict_items([('vendor', 'Cisco'), ('DeviceModel', 'Nexus 9000'), ('OS', 'NX-OS'), ('Location', 'RTP')])
```

.keys() method

```
>>> switch.keys()
dict_keys(['vendor', 'DeviceModel', 'OS', 'Location'])
```

.values() method

```
>>> switch.values()
dict_values(['Cisco', 'Nexus 9000', 'NX-OS', 'RTP'])
```

The last dictionary methods you will cover are pop(), update() and clear().

- **.pop():** **Removes a specified key** from a dictionary and returns that key's value. If the key is not found and no default value is given, a KeyError is raised.
- **.update():** **Merges two dictionaries**. In the example, dictionary switch1 is merged into switch. If a key in switch1 is not in switch, it will be created. If both dictionaries have the same key, the key value in switch will be updated to the new value. A dictionary cannot contain multiples of the same key.
- **.clear():** Clears the dictionary and makes it empty.

.pop() Method

```
>>> switch.pop("OS")
'NX-OS'
>>> switch
{'vendor': 'Cisco', 'DeviceModel': 'Nexus 9000', 'Location': 'RTP'}
```

.update() Method

```
>>> switch1 = {"OS": "NX-OS", "ports": 48}
>>> switch
{'vendor': 'Cisco', 'DeviceModel': 'Nexus 9000', 'Location': 'RTP'}
>>> switch.update(switch1)
>>> switch
{'vendor': 'Cisco', 'DeviceModel': 'Nexus 9000', 'Location': 'RTP', 'OS': 'NX-OS', 'ports': 48}
```

.clear() Method

```
>>> switch.clear()
>>> switch
{}
```

Nested Data Types

List = [List , List , Dict , String]

List = [["IOS-XE", "IOS-XR"], ["R1", "R2"], {"OS": "IOS-XE", "Admin": True}]

List List Dict

Dict = {"D1": { Dict }, "L1": [List] }

Dict = {"R1": {"Gi01": {"ip": "192.168.5.1", "mask": "255.255.255.0"}, "Int": ["Gi01", "Gi02"]}}

Dict List

Navigating Complex Data Structures

The diagram illustrates the indexing of a list named `net`. The list is defined as `net = [[a,b,c], [e,f,g], {"device":{"hostname":"R1", "interface":"Gi01"}}]`. Above the list, indices 0, 1, and 2 are shown, corresponding to the three elements. Brackets below the list indicate that each element is itself a list or dictionary of length 3. Below the code, a series of Python REPL commands and their outputs are shown, with callouts explaining the indexing process:

```
>>> net = [[a,b,c], [e,f,g], {"device":{"hostname":"R1", "interface":"Gi01"}}]
>>> net[0]
['a', 'b', 'c']
>>> net[1][2]
'g'
>>> net[2]["device"]["interface"]
'Gi01'
```

Callouts for the first example:

- First Object in a List (points to `net[0]`)
- Second Object in a List (points to `net[1]`)
- Third Object in a List (points to `net[1][2]`)
- 2nd Key of Nested Dictionary (points to `net[2]["device"]`)
- 1st Key of 1st Dictionary (points to `net[2]["device"]["interface"]`)

Note: when displaying outputs of the list, each `[]` input is a nest of the input `[]` before it

another example:

The diagram illustrates the indexing of a dictionary named `dict`. The dictionary is defined as `dict = {"os":["IOS-XE", "NX-OS"], "device":{"hostname":"R1", "interface":"Gi01"}}]`. Below the code, a series of Python REPL commands and their outputs are shown, with callouts explaining the indexing process:

```
>>> dict = {"os":["IOS-XE", "NX-OS"], "device":{"hostname":"R1", "interface":"Gi01"}}]
>>> dict["os"][1]
'NX-OS'
>>> dict["device"]["interface"]
'Gi01'
```

Callouts for the second example:

- Key of Outside Dictionary (points to `dict["os"]`)
- Index 1 of List (points to `dict["os"][1]`)
- Key of Outside Dictionary (points to `dict["device"]`)
- Key Inside Device Dictionary (points to `dict["device"]["interface"]`)

Manipulating Strings

String Manipulation

Note: strings are immutable, when we modify a string, it creates a new Python object in memory

Methods to manipulate strings include the following:

- **capitalize()**: Returns a copy of the original string and converts the string's first character to an uppercase letter while making all other characters in the string lowercase letters. If the first character is a number, no letters will be capitalized. The command is `str.capitalize()`.
- **title()**: Converts the first character in each word to uppercase, the remaining characters to lowercase, and returns a new string. If the first character of a word is a number, the next letter will be changed to uppercase. The command is `str.title()`.
- **upper()**: Converts the given string into uppercase and returns the string. The command is `str.upper()`.
- **lower()**: Converts the given string into lowercase and returns the string. The command is `str.lower()`.
- **swapcase()**: Converts all uppercase characters to lowercase, all lowercase letters to uppercase, and returns the string.

```
>>> "welcome to cisco prne class".capitalize()  
'Welcome to cisco prne class'
```

First Character of
String Capitalized

```
>>> "welcome to cisco prne class".title()  
'Welcome To Cisco Prne Class'
```

First Letter of All
Words Capitalized

```
>>> "welcome to cisco prne class".upper()  
'WELCOME TO CISCO PRNE CLASS'
```

All Letters of All
Words Capitalized

```
>>> "Welcome to Cisco PRNE Class".lower()  
'welcome to cisco prne class'
```

All Letters of All
Words Lowercase

```
>>> "Welcome to Cisco PRNE Class".swapcase()  
'wELCOME TO cISCO prne cLASS'
```

Letters of All Words
Swapped Upper
and Lower

String Splitting

split() method returns a list of strings after breaking the given string by the specified separator.

`str.split(separator, maxsplit)`

- **separator** – The string splits at this specified separator. If not provided then any white space is a separator.
- **maxsplit** – Parameter times string will be split. If not provided, then there is no limit.

```
>>> router = "cisco 3800 IOS-XE"
>>> router.split()
['cisco', '3800', 'IOS-XE']
```

```
>>> mac_address = "bc67:98c7:77d4"
>>> mac_address.split(':')
['bc67', '98c7', '77d4']
```

example of maxsplit:

```
>>> ip_address = "192.168.5.5"
>>> ip_address.split(".",1)
['192', '168.5.5']
```

example use cases:

splitting up mac addresses

splitting up IP addresses

Dir function

There are two functions that can be used to find which methods are available for a data type.

The first function is the **dir()** function; this is a built-in function that returns a list of attributes along with any available methods, the method description, and syntax.

```
>>> dir(str)
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__',
 '__eq__', '__format__', '__ge__', '__getattr__', '__getitem__',
 '__getnewargs__', '__gt__', '__hash__', '__init__', '__init_subclass__',
 '__iter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__',
 '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmod__',
 '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__',
 'capitalize', 'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs', 'find',
 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isascii', 'isdecimal', 'isdigit',
 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'join',
 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'replace', 'rfind', 'rindex', 'rjust',
 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title',
 'translate', 'upper', 'zfill']
```

Help function

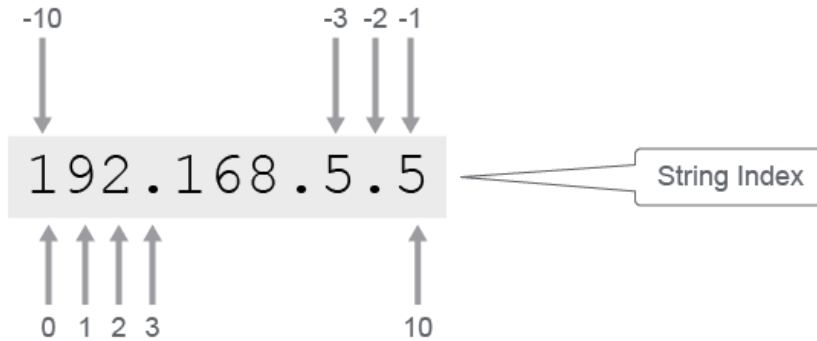
The second function is the **help()** function. The help() function displays documentation of data types, functions, classes, and so on. There are two ways you can use the help() function. The first way is to pass the function an argument, as shown.

```
>>> help(str)
Help on class str in module builtins:

class str(object)
|   str(object='') -> str
|   str(bytes_or_buffer[, encoding[, errors]]) -> str
|
|   Create a new string object from the given object. If encoding or
|   errors is specified, then the object must expose a data buffer
|   that will be decoded using the given encoding and error handler.
|   Otherwise, returns the result of object.__str__() (if defined)
|   or repr(object).
|   encoding defaults to sys.getdefaultencoding().
|   errors defaults to 'strict'.
|
|   Methods defined here:
|
|   __add__(self, value, /)
|       Return self+value.
|
|   __contains__(self, key, /)
|       Return key in self.
```

Modifying Strings (Slicing)

Another way to get specific information from a string is to use the **slice** indexing.



Slice

- Gives a letter from a given index
- `str[index]`

```
>>>ip = "192.168.5.5"
>>>ip[2]
"2"
```

Range Slice

- Slices characters from a given range
- `str[beginning index, ending index, step]`

```
>>>ip = "192.168.5.5"
>>>ip[4:7]
"168"
```

The diagram illustrates string indexing for the string "hostname Router1". The string is displayed in a light gray box. Above the string, two indices are shown with arrows pointing to specific characters: 0 points to the first 'h' and 9 points to the first 'R'. Below the string, the code `>>> hostname = "hostname Router1"` and `>>> hostname[9:]` is shown, followed by the output `'Router1'`.

```
>>> hostname = "hostname Router1"
>>> hostname[9:]
'Router1'
```

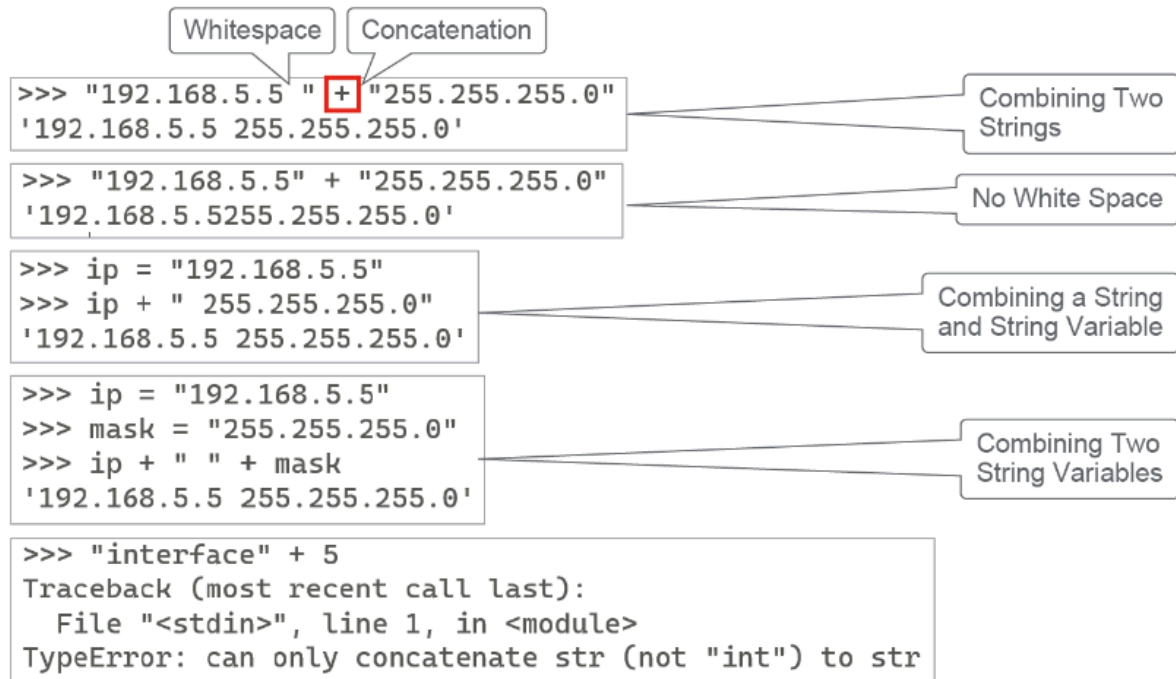
Unlike the `split()` method that splits strings into a list, slice indexing grabs specific characters and then returns them.

String Concatenation (+)

merge two strings

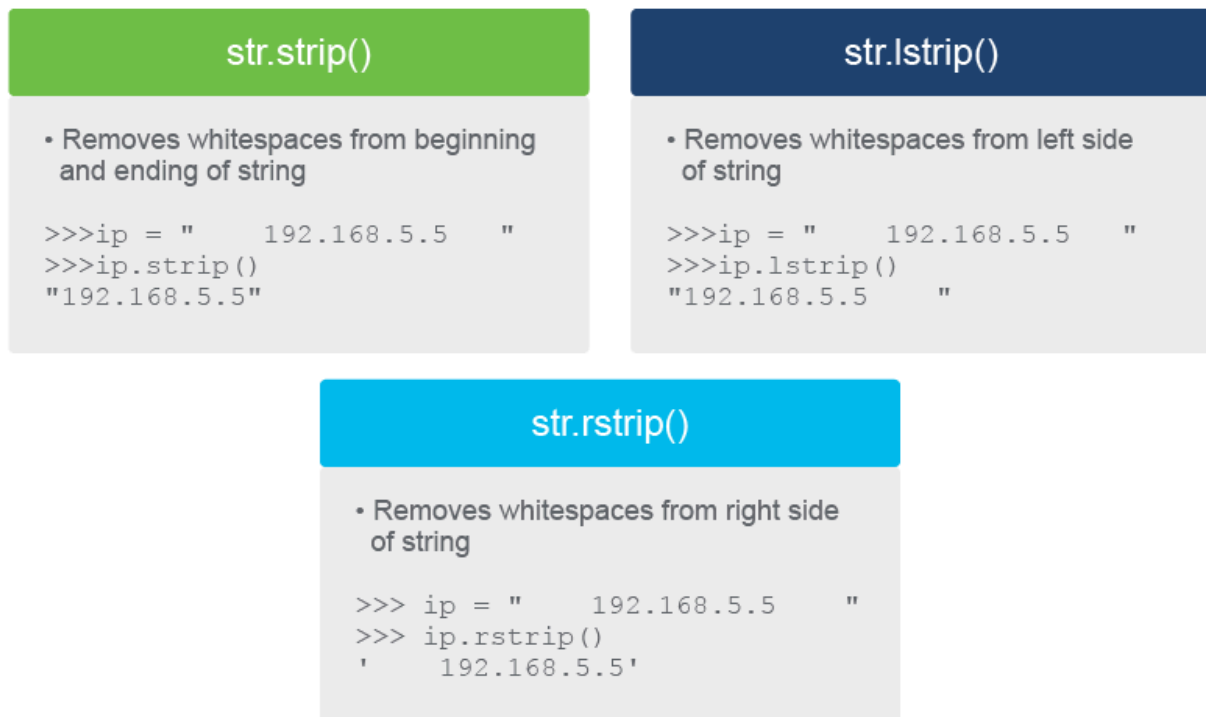
strings are immutable, this creates a new string, a new object in python memory

whitespace added to strings are included



cannot concatenate strings and integers

Whitespace Stripping



Formatting and Templating

As of Python version 3.0, there is a new method of string formatting. Formatting is handled by the `.format()` method.

`.format()` Method

```
>>> x = "router1"  
>>> model = "3800"  
>>> os = "IOS-XE"
```

```
>>> "The model of {0} is {1} with os {2}".format(x, model, os)
```

```
'The model of router1 is 3800 with os IOS-XE'
```

`format()` can print output into columns

```
d = "router1"  
ip = "192.168.5.1"  
mask = "255.255.255.0"
```

```
>>> "{0:^20s}|{1:^20s}|{2:^20s}".format(d, ip, mask)
```

```
'          router1          |          192.168.5.1          |          255.255.255.0          '
```

Instead of putting variables in the `format()` function, they can be put directly into the string:

String Literal(f-String)

```
>>> x = "router1"
>>> model = "3800"
>>> os = "IOS-XE"
```

```
>>> f"The model of {x} is {model} with os {os}"
```

```
'The model of router1 is 3800 with os IOS-XE'
```

Same

.format() Method

```
>>> x = "router1"
>>> os = "IOS-XE"
>>> model = "3800"
>>> "The model of {0} is {1} with os {2}".format(x, model, os)
'The model of router1 is 3800 with os IOS-XE'
```

The following is an example of an f-string:

```
f"The model of {x} is {model} with os {os}"
```

Escape Characters

used to display strings with whitespacing modified (just for that output)

New Line

```
\n
>>> print("Hello\nWorld")
Hello
World
```

Tab

```
\t
>>> print("Hello\tWorld")
Hello    World
```

Backslash

```
\\
>>> print("Hello \\World")
Hello \World
```

Backspace

```
\b
>>> print("Hello \bWorld")
HelloWorld
```

CR

```
\r
>>> print("Hello\rWorld")
World
```

Example 1:

```
>>> 'This is Bill's router.'
```

In Example 1, a single quote is nested inside single quotes. Since you are using a single quote (') to open the string, the string will end at the apostrophe in the word "Bill's", since that is the next single quote ('). To solve this problem, use double quotes (") on the outside of the string, as in Example 2.

Example 2:

```
>>> "This is Bill's router."
```

For the string **This is Bill's router.**, you could use single quotes (') on the outside and use a single quote (') on the inside if you proceed the inside quote with an escape character, as shown in Example 3.

Example 3:

```
>>> 'This is Bill \'s router.'
```

Example 4:

```
>>> 'Router1\n      Interface\n      Gi01'
```

If you want the string, you are printing to display on a new line; you can use (\n). In example 4 Router1, Interface, and Gi01 are all printed on different lines.

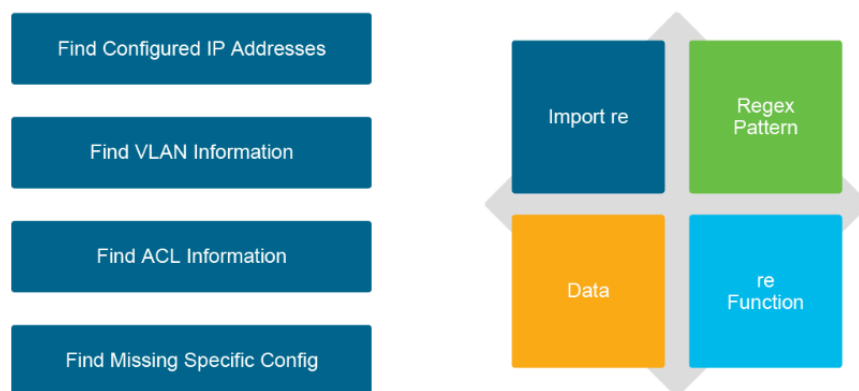
Example 5:

```
>>> print('c:\\>python')  
'c:>'
```

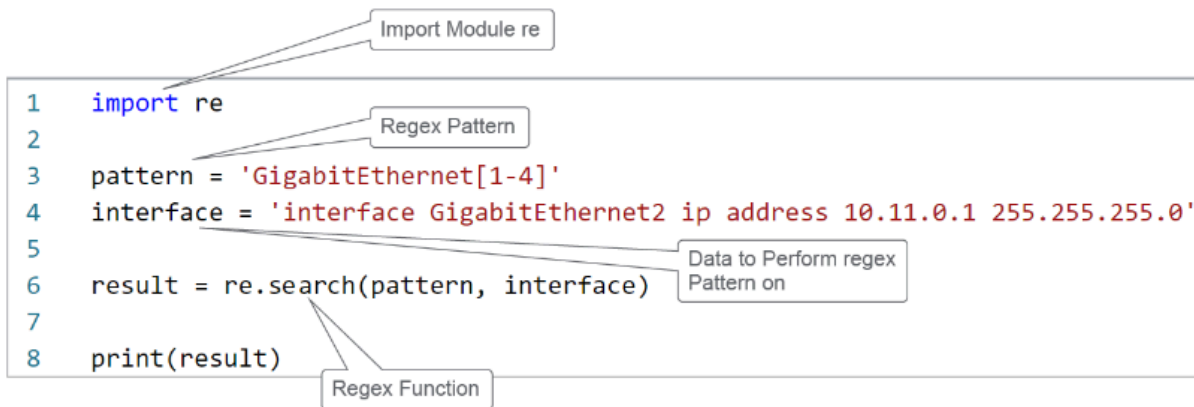
"Escape" characters are useful for adding tabs, new lines, or quotes into your string. But what happens if you want to include a backslash in your string? To print a string similar to the one in example 5 (c:\>python), you will need to use a backslash before the backslash you want to print. This tells the print statement to print the second backslash.

Single Quote	Double Quote
<pre>\ >>> print("Hello \'World\'") Hello 'World'</pre>	<pre>\ >>> print("Hello \"World\"") Hello "World"</pre>
Octal Value	Hex Value
<pre>\000 >>> print("\275") 1/2</pre>	<pre>\xhh >>> print("\x48\x65\x6c \x6c\x6f") Hello</pre>

Regular Expressions



there are 4 parts to using regular expressions:



Line 4's variable 'interface =' could have been the output of a show command or read from a file

Metacharacters

[]	• Set of Characters • [abc], [a-z], [a-zA-Z], [168], [0-9], [1-9][0-9], [^trs]
.	• Any Single Character Except New Line Character • 'P..E'
^	• Starts With • '^PRNE'
\$	• Ends With • 'class\$'

- Square Brackets []: Match any character within the brackets.
 1. [abc] – Matches any character a, b, or c
 2. [a-z] – Matches any lowercase character a through z
 3. [a-zA-Z] – Matches any character lower or uppercase a through z
 4. [168] – Matches any digit 1, 6, or 8
 5. [0-9] – Matches any digit 0 through 9
 6. [1-9][0-9] – Matches any digit 10 through 99
 7. [^trs] – Does not match any of the characters that follow the carat inside the brackets
- Period (.): Matches any character besides a new line character.
 1. 'P..E' – Matches 'PRNE', 'PrnE', or 'PopE'
- Carat (^): Matches strings that start with the characters following the carat.
 1. '^PRNE' – String must start with 'PRNE'
- Dollar sign (\$): Matches strings that end with the characters preceding the dollar sign.

1. 'class\$' – String must end with 'class'

*	• Zero or More Occurrences • [0-9]*
+	• One or More Occurrences • [a-zA-Z]+
?	• Zero or One Occurrences • [1-9]?
{ }	• Specified Number of Occurrences • [abc]{2}

- Star (*): Matches zero or more occurrences of the specified value(s).
 1. [0-9]* – String can have 0 through 9 zero or more times. The string 'Router1' would match because of the '1'.
- Plus (+): Matches one or more occurrences of the given value(s).
 1. [a-zA-Z]+ – The string can have a lowercase or uppercase a through z one or more times. The string '192.168' would not find a match but 'PRNE' would match.
- Question mark (?): Matches zero or one occurrence of the given value(s).
 1. [1-9]? – This would match the string 'Router1' and 'Router'.
- Curly brackets ({}): Matches a specified number of occurrences of the value(s).
 1. [abc]{2} – The string must match two of the preceding characters. For example, the string 'ab567' would match because of the 'ab' and the string 'ca567' would match, but the string 'a567b' would not match because the two characters are not together.

()	• Capture and Group • Switch(1 2 3)+
\	• Used to Escape Various Characters • \ \$, \.
	• Either or • AB 12

- Parenthesis (): Used to group sub-patterns

1. Switch(1|2|3)+ – Will match any string with the characters 'Switch' followed by 1 or more of the following digits 1, 2, or 3. The string 'Switch1' will match, the string 'Switch11' will match, and the string 'Switch1223' will match, but the string 'Switch4' will not.
- Escape (\): Used to escape various characters, allowing them to be matched
 1. \\$ – Matches the \$ exactly so it is not used as a metacharacter.
 2. \. – Matches the . exactly so it is not used as a metacharacter.
- Vertical bar (|): Used for either or
 1. AB|12 – For a string to match it can either have the characters AB or 12 in the string. So, the string 'SwitchAB' would match and the string 'Switch12' would match, but the string 'SwitchA' would not.

Test regular expressions at <https://regexr.com/>

search(pattern, data)

- Looks For First Match
- Returns Match Object or None

findall(pattern, data)

- Returns a List of Strings Containing All Matches

split(pattern, data)

- Splits the String Where There is a Match
- Returns a List of Strings Where the Split Occurred

There are multiple methods available for working with regular expressions such as **search()**, **findall()**, and **split()**. To learn about other methods, visit <https://docs.python.org/3/howto/regex.html>.

The **search()** method has two parameters. The first parameter is the regular expression pattern you want to match. The second parameter is the data you are checking with the pattern.

```
data = 'abc123cba321'  
pattern = '[abc][123]'  
result = re.search(pattern, data)  
<re.Match object; span=(2, 4), match='c1'>
```

In the example, you check the string 'abc123' stored in the variable `data` with the pattern `[abc][123]`. The pattern will find a match if the string has a character a, b, or c followed by a digit 1, 2, or 3. When you use the search method, it will return an object of the first match, or, if there is no match, it will return 'None'. Since the `search()` method found a match, it returned: `<re.match object; span=(2,4), match='c1'>`. The `span` indicates the beginning and ending index the match was found at and the `match` shows the matching string.

The **`findall()`** method searches your data for a pattern and returns all matches in a list. Like the `search()` method, the **`findall()`** method takes as parameters a pattern and a data source.

```
data = 'abc123cba321'  
pattern = '[abc][123]'  
result = re.findall(pattern, data)  
['c1', 'a3']
```

In the example, you search the data stored in the variable `data` with the following pattern `[abc][123]`. The **`findall()`** method returns the following result, `['c1', 'a3']`.

The **`split()`** method splits the data any place there is a pattern match and returns a list of strings adjacent to the splits. The **`split()`** method also takes as parameters a pattern and a data set.

```
data = 'abc123cba321'  
pattern = '[abc][123]'  
result = re.split(pattern, data)  
['ab', '23cb', '21']
```

In the example, the string 'abc123cba321' stored in the variable `data` is split where there is a pattern match. The pattern matches 'c1', and 'a3' and the **`split()`** method returns the result `['ab', '23cb', '21']`.

Regular expressions

#####

The 1 in `match.group(1)` represents the first parenthesised subgroup. In your case the first of one.

This is well described in the official documentation for `re.MatchObject.group()` and best illustrated with an example:

```
>>> m = re.match(r"(\w+) (\w+)", "Isaac Newton, physicist")
>>> m.group(0)    # The entire match
'Isaac Newton'
>>> m.group(1)    # The first parenthesized subgroup.
'Isaac'
>>> m.group(2)    # The second parenthesized subgroup.
'Newton'
>>> m.group(1, 2) # Multiple arguments give us a tuple.
('Isaac', 'Newton')
```

String Methods

`str.count(sub, start, end)`

```
>>> x = "123 123 123 123"
>>> x.count("1")
4
```

```
>>> x = "123 123 123 123"
>>> x.count("1", 4, 10)
2
```

`str.find(sub, start, end)`

```
>>> x = "123 123 123 123"
>>> x.find("123")
0
```

```
>>> x = "123 123 123 123"
>>> x.find("123", 4, 7)
4
```

`str.startswith(prefix, start, end)`

```
>>> ip = "192.168.5.1"
>>> ip.startswith("192")
True
```

```
>>> ip = "192.168.5.1"
>>> ip.startswith("192", 4, 7)
False
```

the `count()`, `find()`, and `startswith()` functions can use start and stop parameters:

```
x.count("1", 4, 10)
```

4 is start
10 is stop parameter

Note: replace "str" with the *variable*

str.count()
str.find()
str.startswith()

example use case:

find IP addresses with starting octet:

```
>>> ip = "192.168.5.1"
>>> ip.startswith("192")
True
```

```
>>> ip = "192.168.5.1"
>>> ip.startswith("192", 4, 7)
False
```

Mac Addresses

regex to pull mac addresses:

```
show_arp = net_connect.send_command('show arp')
print(show_arp)
```

```
mac_list = re.findall(r'([0-9a-f]{4}\.){2}[0-9a-f]{4}', show_arp)
print(mac_list)
```

ex.

```
Internet 10.12.0.1 - 0050.56ac.ba7b ARPA GigabitEthernet2
Internet 10.12.0.2 205 0050.56ac.0e60 ARPA GigabitEthernet2
Internet 10.13.0.1 - 0050.56ac.9ea0 ARPA GigabitEthernet3
Internet 10.13.0.3 169 0050.56ac.0e60 ARPA GigabitEthernet3
Internet 10.254.0.1 - 0050.56ac.93be ARPA GigabitEthernet4
Internet 10.254.0.10 13 0050.56ac.e4ef ARPA GigabitEthernet4
['0050.56ac.ba7b', '0050.56ac.0e60', '0050.56ac.9ea0', '0050.56ac.0e60', '0050.56ac.93be', '0050.56ac.e4ef']
student@workstation:~/lab_works
```

Normalize: Another strategy would be to take the item in question and put it into a normalized form. A good example would be for dealing with MAC addresses, which can be presented in CLIs with colons (':'), or hyphens ('-') every two bytes. Sometimes they are even represented with dots ('.') every four bytes. Normalizing by changing all characters to lower case, then removing the hyphens, colons, or dots, helps to make all MAC address forms 'equal', so that they can be compared against one another, no matter where they came from.

```
mac_addr = mac_addr.lower() # get mac address into lower case
```

```
mac_addr = mac_addr.replace(':', '') # strip out colons
mac_addr = mac_addr.replace('-', '') # strip out dashes
mac_addr = mac_addr.replace('.', '') # strip out dots
```

change_mac_notation module

```
import re
import netmiko
import change_mac_notation
#show arp command
show_arp = net_connect.send_command('show arp')
#pull mac addresses only
mac_list = re.findall(r'((?:[0-9a-f]{4}\.){2}[0-9a-f]{4})', show_arp)
#change notation for output from mac_list
print(change_mac_notation.change_notation(mac_list[0], '-'))
#replace '-' with ':' or any other notation
```

Convert MAC addresses from EUI-48 to EUI-64

```
import re
import netmiko
import change_mac_notation !not needed?
import eui48to64
#show arp command
show_arp = net_connect.send_command('show arp')
#pull mac addresses only
mac_list = re.findall(r'((?:[0-9a-f]{4}\.){2}[0-9a-f]{4})', show_arp)

print(eui48to64.convert(mac_list))
```

Describing Conditionals, Loops, and Operators

Whitespace and PEP 8

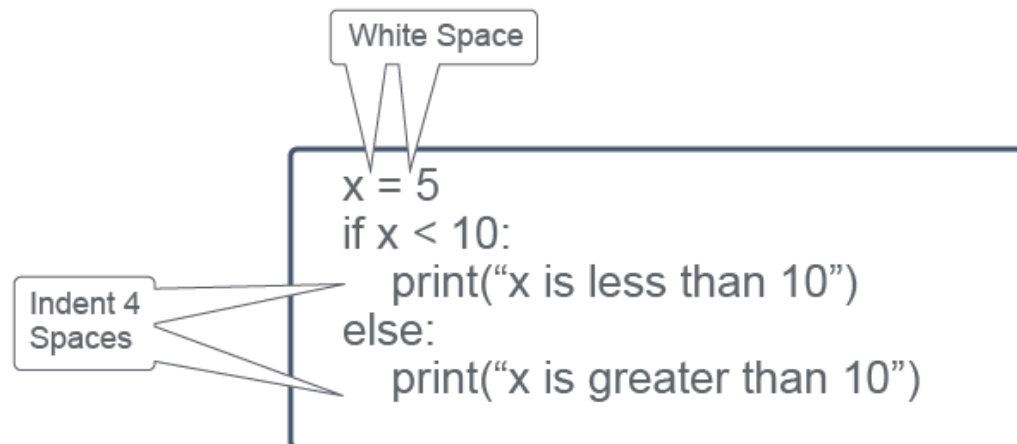
PEP = Python Enhancement Proposal

PEP 8 is a style guide for Python Code

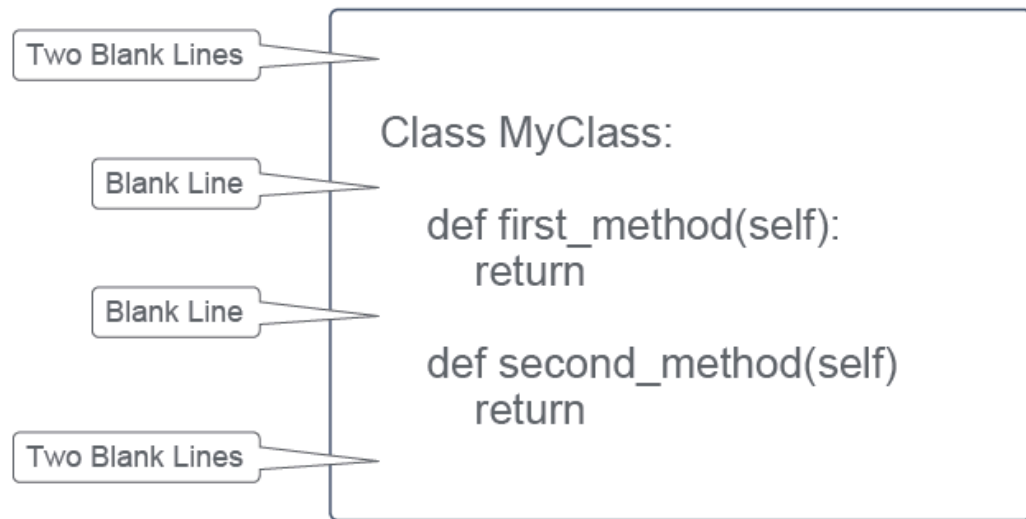
some style guide examples:

Maximum Line Length • Limit all lines to 79 characters.	Line Break • Line break should be before a binary operator. <pre>income = (gross_wages + taxable_interest)</pre>	Blank Lines • Surround top-level functions and class definition with two blank lines. • Method definitions inside a class are surrounded with a single blank line.
Imports • Imports should be at the top of a file. • Imports should usually be on separate lines.	Naming • Never use characters l (lowercase letter el), O (uppercase oh), I (upper-case letter eye) as single character variable names. • Variable and function names should be lowercase with underscore separating words.	Tab or Spaces • Always use spaces unless you are trying to be consistent with code that uses tabs.

PEP 8: Whitespace



PEP 8: Blank Lines



PEP 8: Checking Code for Conformity

```
$ pip install pycodestyle
```

```
$ pycodestyle filename.py
```

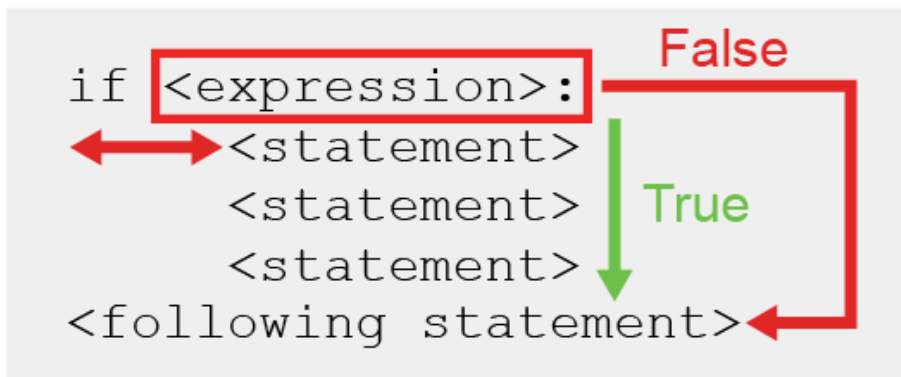
Output from Pycodestyle

```
warning.warn(  
pep8check.py:4:2: E225 missing whitespace around operator  
pep8check.py:5:2: E225 missing whitespace around operator  
pep8check.py:6:5: E221 multiple spaces before operator  
pep8check.py:8:1: E112 expected an indented block  
pep8check.py:10:1: W391 blank line at end of file
```

Conditionals

if <expression> is true, indented lines will iterate

if the expression is false, it will move to the next statement not indented 4 spaces



```
1  x = True  
2  if x:  
3      print("x is True")  
4  print("This code is outside the if")
```

```
C:\PRNE>python if.py  
x is True  
This code is outside the if
```

```
1  x = False  
2  if x:  
3      print("x is True")  
4  print("This code is outside the if")
```

```
C:\PRNE>python if.py  
This code is outside the if
```

To write the first line of your if statement, you need the expression to be evaluated, followed by a colon (:). To group your lines of code that follow the expression, **indent each statement by 4 spaces.**

else Statement

```
if <expression>:
    <statement>
else:
    <statement>
```

Flowchart illustrating the execution of an if-else statement. The flow starts at the 'if <expression>' condition. If the expression is True (indicated by a green arrow), the flow proceeds to the first <statement>. If the expression is False (indicated by a red arrow), the flow proceeds to the 'else:' block and then to its <statement>.

elif statements act as sections of if statements

elif Statement

```
if <expression>:
    <statement>
elif <expression>:
    <statement>
elif <expression>:
    <statement>
else:
    <statement>
```

Flowchart illustrating the execution of an if-elif-else statement. The flow starts at the 'if <expression>' condition. If it is True (blue arrow), the flow proceeds to the first <statement>. If it is False, the flow proceeds to the 'elif <expression>' condition. If that is True (blue arrow), the flow proceeds to the second <statement>. If that is False, the flow proceeds to the next 'elif <expression>' condition. If that is True (blue arrow), the flow proceeds to the third <statement>. If that is False, the flow proceeds to the 'else:' block and then to its <statement>.

```
1 x = False
2 y = True
3
4 if x:
5     print("x is True")
6 elif y:
7     print("y is True")
8 else:
9     print("x and y are not True")
```

```
C:\PRNE>python elif.py
y is True
```

```
1 x = False
2 y = False
3
4 if x:
5     print("x is True")
6 elif y:
7     print("y is True")
8 else:
9     print("x and y are not True")
```

```
C:\PRNE>python elif.py
x and Y are not True
```

example of using conditionals

```
x = 8
if x < 8:
    print("the value is less than 8")
elif x > 8:
    print("The value is greater than 8")
else:
    print("The value is 8")
```

```
C:\PRNE>python conditionals.py
The value is 8
```

Create notation for subnets:

```
ip = '192.168.0.1'
```

```
subnet = '255.255.255.0'
```

```
if subnet.endswith('.0'):
```

```
    subnet_slash_notation = '/24'
```

```
    print(ip + subnet_slash_notation)
```

```
else:
```

```
    print('Subnet slash notation could not be determined')
```

Operators

Operators are divided into the following groups:

- Arithmetic
- Assignment
- Comparison
- Logical
- Identity
- Membership

Arithmetic Operators

Arithmetic operators include addition, subtraction, multiplication, division, and exponentiation.

Addition <code>x = 10</code> <code>y = 5</code> <code>x + y</code> 15	Subtraction <code>x = 10</code> <code>y = 5</code> <code>x - y</code> 5	Multiplication <code>x = 10</code> <code>y = 5</code> <code>x * y</code> 50
Division <code>x = 10</code> <code>y = 5</code> <code>x / y</code> 2.0	Modulo <code>x = 11</code> <code>y = 5</code> <code>x % y</code> 1	
Exponentiation <code>x = 10</code> <code>y = 5</code> <code>x ** y</code> 100000	Floor Division <code>x = 11</code> <code>y = 5</code> <code>x // y</code> 2	

Assignment Operators

Assignment operators are used to assign values to variables.

Assignment <code>x = 5</code>	Addition Assignment <code>x += 5</code> <code>x = x + 5</code>	Subtraction Assignment <code>x -= 5</code> <code>x = x - 5</code>
Multiplication Assignment <code>x *= 5</code> <code>x = x * 5</code>	Division Assignment <code>x /= 5</code> <code>x = x / 5</code>	Modulo Assignment <code>x %= 5</code> <code>x = x % 5</code>
Floor Assignment <code>x //= 5</code> <code>x = x // 5</code>	Exponentiation Assignment <code>x **= 5</code> <code>x = x ** 5</code>	

Comparison Operators

Comparison operators are used to compare values and return either a Boolean True or False.

Equal x = 5 y = 3 x == y False	Not Equal x = 5 y = 3 x != y True	Greater Than x = 5 y = 3 x > y True
Less Than x = 5 y = 3 x < y False	Greater Than or Equal To x = 5 y = 3 x >= y True	Less Than or Equal To x = 5 y = 3 x <= y False

Logical Operators

Logical operators are “and,” “or,” and “not” and return a Boolean True or False once the expression is evaluated.

and x = 4 x > 3 and x > 6 False	or x = 4 x > 3 or x > 6 True	not x = 4 not(x > 3 or x > 6) False
---	--	---

Identity Operators

Identity operators check to see if two values are located in the same part of memory.

Identity operators are not looking to see if the two variables have the same value but if the two variables point to the same memory location. Let's take a closer look at the identity operators is and is not.

is x = ["router1", "router2"] y = ["router1", "router2"] z = x x is z True x is y False	is not x = ["router1", "router2"] y = ["router1", "router2"] z = x x is not z False x is not y True
---	---

x is y = false because although the two lists look identical, they sit in two different memory

locations

-if you want to check if they have the same list, use ==

Membership Operators

Membership operators check to see if a variable or value is found in a string, list, tuple, set, or dictionary.

in

```
x = ["router1", "router2"]  
"router1" in x  
True
```

not in

```
x = ["router1", "router2"]  
"router3" not in x  
True
```

Loops

**For
Loop**

```
for <var> in <iterable>:  
    <statement>
```

This iterable can be a **string, list, dictionary, tuple, or set**.

```
for i in "string":  
    print(i)
```

```
C:\PRNE>python forloop.py  
s  
t  
r  
i  
n  
g
```

in this example, each letter of "string" is assigned to (i)

The line of code takes each member of the iterable and assigns it to the variable one loop at a time.

In this example, `i` is a variable that will hold each object's member that we iterate through to be used in the code block. The result is that each letter of "string" is stored in `i` and then printed. The "for" loop will exit after the last member of the iterable is reached.

While Loop

```
while <expression>:  
    <statement>
```

```
i = 4  
while i > 0:  
    print(f'The value of i is {i}')  
    i -= 1
```

```
C:\PRNE>python whileloop.py  
The value of i is 4  
The value of i is 3  
The value of i is 2  
The value of i is 1
```

Note: an assignment operator, subtraction assignment, was used to subtract 1

The basic format of a "while" statement follows:

```
while <expression>:
```

We start the statement with the keyword "while", followed by an expression. The

expression uses a variable that is set to a specific value before the loop. The expression is checked, and if it evaluates to True, the code block is executed. Typically, the variable used in the expression is modified inside the code block. After each iteration, the expression is checked, and the code block **will continue to execute until the expression evaluates to False**. Let's look at the example.

```
>>> i = 4
>>> while i > 0:
>>>     print(f'The value of i is {i}')
>>>     i -= 1
```

Before the "while" loop, the variable `i` was assigned the value of 4. The "while" loop evaluates the expression `i > 0`, so for as long as `i` is greater than 0, the loop will execute. The code block inside the "while" loop prints a string, then subtracts 1 from `i`. As long as `i > 0`, the loop will execute again.

while loop example:

```
1  import netmiko
2
3  last_octet = 1
4  while last_octet <= 15:
5      ip_address = '10.254.0.' + str(last_octet)
6      print(ip_address)
7      last_octet += 1
```

```
student@workstation:~/lab_work$ python while_loop.py
10.254.0.1
10.254.0.2
10.254.0.3
10.254.0.4
10.254.0.5
10.254.0.6
10.254.0.7
10.254.0.8
10.254.0.9
10.254.0.10
10.254.0.11
10.254.0.12
10.254.0.13
10.254.0.14
10.254.0.15
```

Loops with Lists, Dicts, and Ranges

Loops with Lists

Loop with List

```
>>> devices = ["router1", "router2", "router3", "switch1"]
>>> for device in devices:
...     print(device)
...
router1
router2
router3
switch1
```

Let's start with how to iterate over a list using a "for" loop shown in the example:

```
for device in devices:
    print(device)
```

In the example, there is a list that contains the items "router1," "router2," "router3," and

"switch1". To access each of these items, you can use a "for" loop. There is a variable named device in the "for" loop to which each item will be assigned. The variable is used in the print statement to print each device in the list.

Loops with Dictionaries

Looping through dictionaries is not as easy as lists. In dictionaries, you can abstract keys, values, or key-value pairs (known as an item) to use in your code block. A dictionary has methods that can be used to abstract each of these items. Let's examine each of these, starting with keys.

Loop with Dictionary

```
device = {"device": "router", "model": "3800", "os": "IOS-XE"}  
  
for keys in device.keys():  
    print (keys)
```

device
model
os

Result

```
for value in device.values():  
    print (value)
```

router
3800
IOS-XE

Result

To get the keys from a dictionary, use the **keys()** method. The **keys()** method saves the dictionary keys in an iterable that can then be used in a "for" loop. The example shows a "for" loop with a variable named keys. The iterable used in the for loop is **device.keys()**. You can then use the print statement to print the values stored in the keys variable.

To get the values from a dictionary, use the **values()** method. The **values()** method saves the value of each key in an iterable. The example shows a "for" loop with a variable named values. The iterable used in the for loop is **device.values()**. You can then use the print statement to print the values stored in the values variable.

To get the key-value pairs from the dictionary, you use the **.items()** method. The **.items()** method saves the key-value pairs in tuples. Tuples are similar to lists, but the

items in the tuple cannot be changed.

Loop with Dictionary

```
device = {"device": "router", "model": "3800", "os": "IOS-XE"}

for item in device.items():
    print(item)
```

```
('device', 'router')
('model', '3800')
('os', 'IOS-XE')
```

Result

The example has a dictionary with the key value pairs of "device": "router," "model": "3800," and "os": "IOS-XE". You can use a “for” loop to iterate over the items to print in the print statement. The example uses a “for” loop with the variable name `item` and the iterable used in the “for” loop is `device.items()`. `Device.items()` takes each key-value pair and saves it in a separate tuple that you can then use to execute the code inside the “for” loop.

Loops with Ranges

Up to this point, you have learned how to use a loop to iterate through all the items in a list or dictionary. What if you only want certain items in a list or dictionary? Maybe you are using an API that returns data in a JSON format. You convert the JSON format to a Python dictionary and you only want certain items. You can achieve this using the `range()` function. The `range()` function allows you to generate a series of numbers within a given range. This given range depends on how many arguments you pass to it.

Loop with Ranges

`range(start, stop, step)`

```
device = ["router1", "router2", "router3", "switch"]  
  
for i in range(2, len(device)):  
    print(device[i])
```

```
router3  
switch
```

Result

The command for **range()** follows.

`range(start, stop, step)`

- start: number to start
- stop: number to stop. The stop number is not included in the count. So, if 1 is the start and 8 is the stop, the range will be 1-7.
- step: the difference between one number to the next

A simple example follows.

```
for i in range(1, 4):  
    print("router" + str(i))  
"router1"  
"router2"  
"router3"
```

In this example, the range is 1 to 4; each value is stored in the variable `i`. It then prints "router" along with the number stored in `i`.

Another way to use **range()** is with **len()**. The **len()** function returns the total number of items in an object. This can be used as the stop number, so it will always be the last item in the object.

```
for i in range(2, len(devices)):
    print(device[i])
```

In this example, you want to print each item starting with index 2 through the last item. To do this, use the `range()` function with a starting number of 2. If you don't know how many items are in the object, you can use the `len()` function to determine the stop number. If the total number of items is less than two, using the `len()` function will return an error.

Loops with Conditionals

```
1  device = ["Router1", "Router2", "Router3", "Router4"]
2
3  for i in range(len(device)):
4      if device[i] == "Router3":
5          print("This is inside the if-statement")
6      print("This is outside the if-statement")
```

```
C:\PRNE>python LoopConditional.py
This is outside the if-statement
This is outside the if-statement
This is inside the if-statement
This is outside the if-statement
This is outside the if-statement
```

Conditionals will execute a block of code once if a condition is met and loops will continue to execute a block of code as long as a condition is met. At times you will want to combine these two. The example uses a for loop to iterate over a list with an if-statement inside the for loop checking for a specific condition.

Line 1 of the code is a list of routers. The code starting at line 2 uses a for loop to iterate over each item in the list (device) and execute the block of code below it. The key is that you only want part of the code to execute if a condition is met. To do this you use an if-statement.

For each item in the list device, the if-statement is will be evaluated to see if device[i] is equal to "Router3". If this condition is False, the code block under the if-statement is

skipped and the print statement in line 6 is executed. The for loop then assigns the next item in the list to i and the if statement is evaluated again. When or if the if-statement evaluates to True, the print statement in line 5 is executed and then the loop continues until there are no more items in the list.

You can use conditionals inside of loops to help determine when a block of code runs. When using conditionals, there might be times where, if a condition is met inside a loop, we break completely out of the loop or want to skip executing the block of code below the loop and start the loop over. In this case, there are two statements you can use: the break and continue statements.

Break

```
device = ["Router1", "Router2", "Router3", "Router4"]
for i in range(len(device)):
    if device[i] == "Router3":
        break
    print(device[i])
```

```
C:\PRNE>python break.py
Router 1
Router 2
```

The break statement terminates the loop containing it. Let's look at an example.

```
device = ["Router1", "Router2", "Router3", "Router4"]
for i in range(len(device)):
    if device[i] == "Router3":
        break
    print(device[i])
```

In the example, we are iterating over a list of devices, and when device[i] == "Router3," it will break out of the loop. This script results in only Router1 and Router2 being printed.

Continue

```
device = ["Router1", "Router2", "Router3" , "Router4"]
for i in range(len(device)):
    if device[i] == "Router3":
        continue
    print(device[i])
```

```
C:\PRNE>python continue.py
Router 1
Router 2
Router 4
```

The continue statement is used to skip the rest of the code inside a loop for the current iteration. The loop does not terminate but continues with the next iteration. Let's look at an example.

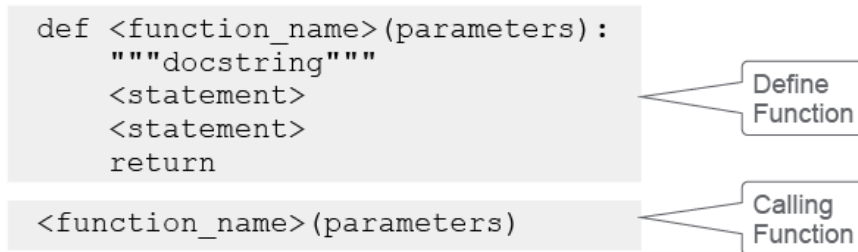
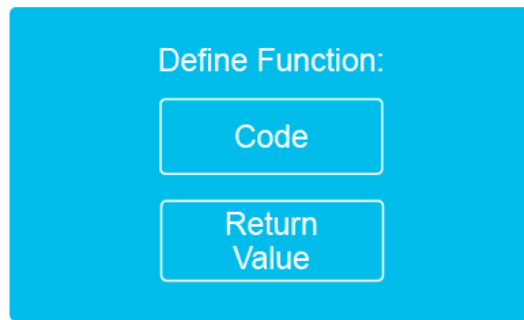
```
device = ["router1", "Router2", "Router3", "Router4"]
for i in range(len(device)):
    if device[i] == "Router3":
        continue
    print(device[i])
```

In the example, we are iterating over a list of devices, and when `device[i] == "Router3"` it will continue. Continue means it will skip the print statement and start the next iteration. This script results in only Router1, Router2, and Router4 being printed.

Exploring Classes, Methods, Functions, Namespaces, and Scopes

Functions

self-contained block of code that executes a specific task



A function is made up of a few components:

1. def Function name
2. Code
3. Return value (optional)

The syntax for defining each of these components is below:

1. def <function_name>([<parameters>]):
2. <statement(s)>
3. return(value)

The **def keyword informs Python that a function is being defined**. The <function_name> is an identifier that names the function. Following the name is the <parameters>; this is optional, but it is used to pass data to the function statements. The colon (:) represents the end of the function header.

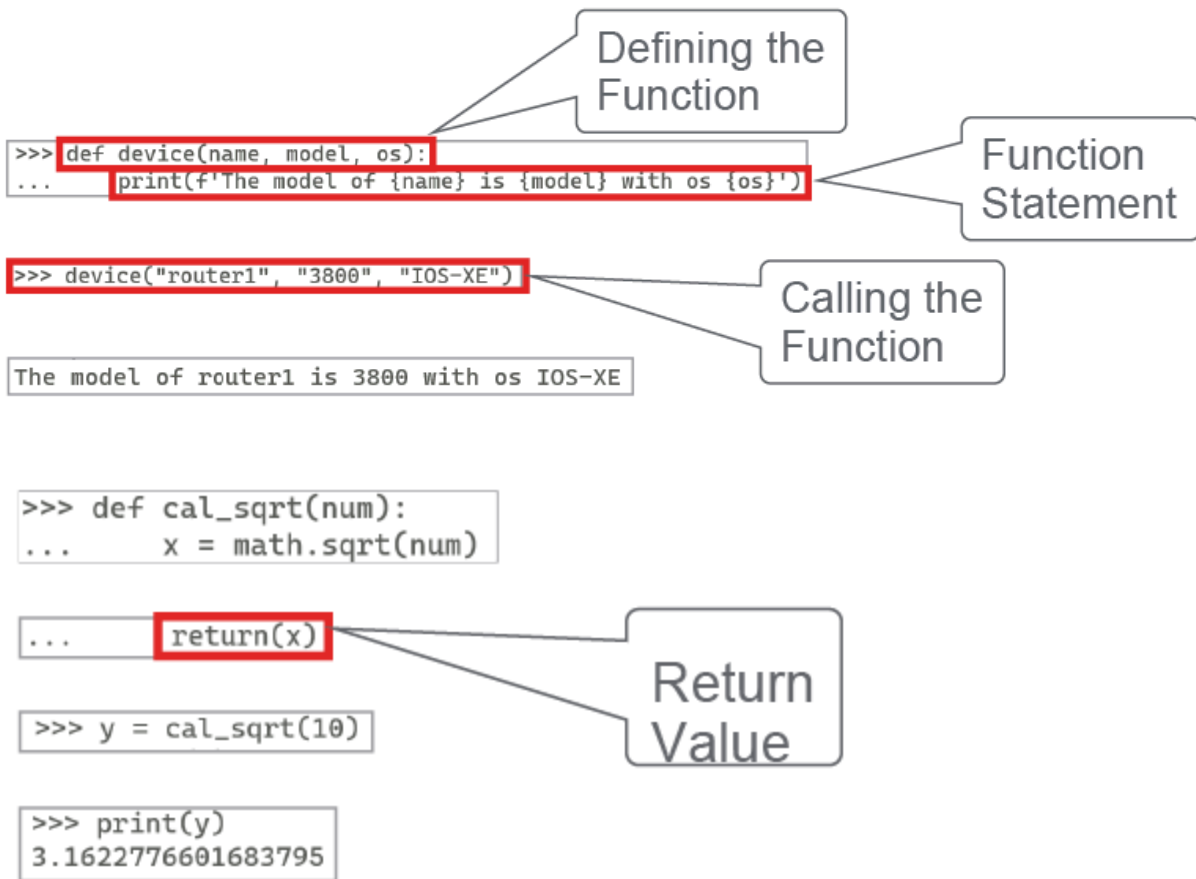
The <statement(s)> is a block of valid code executed when the function is called. The body of the function is defined by indenting four spaces.

The return(value) is optional, but it allows for the function to return values to the code that is called.

Now that you know how to define a function, how do you use it? Anywhere in your code, you want to use the function; you need to call it. The syntax for calling a function is as follows:

<function_name>([<arguments>])

The <arguments> are the values passed into the function that correspond to the <parameters> in the Python function definition.



These examples show how to define a function with and without a return statement. The top example is defining a function named `device` that will take parameters called `name`, `model`, and `os`. The body of the function has one statement that prints a string using the parameters passed to the function. There will be no value returned to the calling code.

To call the function, use the function name and the parameters you want to pass to it.

Note

You must enter the exact number of parameters and in the correct order in using parameters this way. There are other ways to accept parameters into a function, but they are not covered in this training.

The bottom example is using the `return` statement to return the value of `x` to the calling code. In this example, you call the function, and the returned value of `x` gets assigned to `y`. The value for `y` is then printed.

Code Commenting

Code Commenting: Single and Multiline

```
1 import csv
2 from pprint import pprint
3
4 # data is a variable holding
5 # the csv values that will be read
6 data = [['Hostname', 'Vendor', 'Model', 'Location'],
7         ['sw10', 'Cisco', '3800', 'Miami'],
8         ['sw11', 'Cisco', '3650', 'Atlanta']]
9
10 # Next we will open a file
11 with open('newdevices.csv', 'w') as f:
12     print(data, file=f)
```

Multi-Line Comment

Single-Line Comment

Code Commenting: Docstring

A docstring is a string literal that occurs as the first statement in a module, function, class, or method definition.

```
1 class ComplexNumber:
2
3     """
4     This is a class for complex numbers.
5     It takes in two variables r and i.
6     """
7
8     def __init__(self, r=0, i=0):
9         self.real = r
10        self.imag = i
11
12    def get_data(self):
13        """This function adds a real and imaginary number."""
14        print(f'{self.real}' + {self.imag})
```

Docstring

Python has a built-in concept called “docstrings,” described in PEP 257. A docstring is a string literal that occurs as the first statement in a module, function, class, or method and defines their function. Classes will be discussed in an upcoming section.

Docstrings follow these rules:

- Docstrings are surrounded by ""triple double quotes""."
- A docstring can be either a single line or a multiline comment.
- Unless the entire docstring fits on a line, place the closing quotes on a line by themselves.
- A docstring begins with a capital letter and ends it with a period.

The example shows a single and a multiline docstring. The multiline docstring is used after the class declaration. Note that the comments and the quotes are on different lines. A single-line docstring is used in the function where the quotes and comment are on the same line. The docstring's purpose is to provide users with an overview of the object. It should be concise enough to be easy to maintain but detailed enough for users to understand the purpose and how to use the object.

Namespaces and Scopes

A namespace is a collection of currently defined names and the objects that the names reference. When you execute a Python program, the interpreter will create namespaces when necessary and delete them when they are no longer needed. At any given time, you can have many namespaces created.

Code

```

1  a, b, c = 1, 2, 3
2
3  def f1():
4      a, b, c = 10, 20, 30
5      print(f'Outer function f1() variables: a is {a}, b is {b}, c is {c}')
6
7      def f2():
8          a, b, c = 100, 200, 300
9          print(f'Inner function f2() variables: a is {a}, b is {b}, c is {c}')
10
11         f2()
12
13     f1()
14     print(f'Global variables: a is {a}, b is {b}, c is {c}')

```

The diagram illustrates the nesting of namespaces. The Global Scope contains the global variables `a, b, c = 1, 2, 3` and the `f1()` function. The `f1()` function defines an Outer Function namespace, which contains the variables `a, b, c = 10, 20, 30` and the `f2()` function. The `f2()` function defines an Inner Function namespace, which contains the variables `a, b, c = 100, 200, 300`. The output of the code is shown in the Output section.

Output

```

Outer function f1() variables: a is 10, b is 20, c is 30
Inner function f2() variables: a is 100, b is 200, c is 300
Global variables: a is 1, b is 2, c is 3

```

In a Python program, there are four types of namespaces, each having different lifetimes:

1. Built-In: Contains the names of all Python's built-in objects. The Python interpreter creates the built-in namespace when it starts up, and they are available always when Python is running. You can list the objects in the built-in namespace with the command `dir(__builtins__)`. This namespace remains in existence until the interpreter terminates.

2. Global: Contains any names defined in the main program or any module loaded with the import statement. This namespace terminates when the interpreter terminates.
3. Enclosing: Contains any names in the outer function of a nested function. This namespace terminates when the function terminates. In the example shown, the enclosed namespace is f1() because f2() is nested inside.
4. Local: Contains any names at the function level. This namespace terminates when the function terminates. In the example shown, the local namespace is f2() or if you had any function in your code that didn't have a function nested inside, it would be considered local.

Each namespace has a scope. The scope is the part of the program in which the name has meaning. When a variable is used, the interpreter looks inside out for a definition. So, the order it will check is local, enclosed, global, and finally built-in.

Referencing the example shown, the process works as follows:

1. The main program defines three variables: a, b, and c. At the end of the script, it has a print statement that prints the values of a, b, and c, and it calls a function name f1().

```
a, b, c = 1, 2, 3
...
f1()
print(f'Global variables: a is {a}, b is {b}, c is {c}')
```

1. When you execute the program, a, b, and c are assigned the value 1, 2, and 3, respectively. At the end of the script, the print statement prints the following: 'Global variables: a is 1, b is 2, c is 3'
2. Above the print statement, the function f1() is called. Function f1() has a nested function named f2().

```
def f1():
    a, b, c = 10, 20, 30
    print(f'Outer function variables: a is {a}, b is {b}, c is {c}')
```

1. In f1() the variable a = 10, b = 20, and c = 30. The print statement in f1() prints the values of a, b, and c. The output of the print statement is as follows: 'Outer function f1() variables: a is 10, b is 20, c is 30'. So which variables a, b, and c are being used here? The variables a, b, and c are defined in the function so those values are used. Python will use the most local variables available regarding functions; Python will always check if that variable is defined inside the function or as an argument before checking a nonlocal scope or the global scope.
2. After the print statement in function f1(), function f2() is called.

```
def f2():  
    a, b, c = 100, 200, 300  
    print(f'Inner function f2() variables: a is {a}, b is {b}, c is {c}')
```

1. In f2() the variables a, b, and c are defined as 100, 200, and 300. The print statement prints the values of a, b, c. The results are as follows: 'Inner function variables: a is 100, b is 200, c is 300'. The variables are defined inside f2() so those are the variables used, but if a, b, or c weren't defined in f2(), Python would check the nonlocal scope f1() first, then the global scope for a variable to use.
- 2.

You can see from this example that the interpreter looks inside out for variable definitions. If you try to call a variable in your main program that is only used in a function, you will raise the `NameError` indicating that the variable has not been defined.

Classes and Methods

In object-oriented programming such as Python, everything you create is an object. An object can have attributes and methods that are linked to it. So how do you know what attributes and methods are linked to the object? **Objects are an instance of a class**. You can think of a class as a blueprint.

Class (Blueprint)

Class: Cisco Switch

Attributes: Hostname
Management_IP
VLAN_ID
VLAN_Name

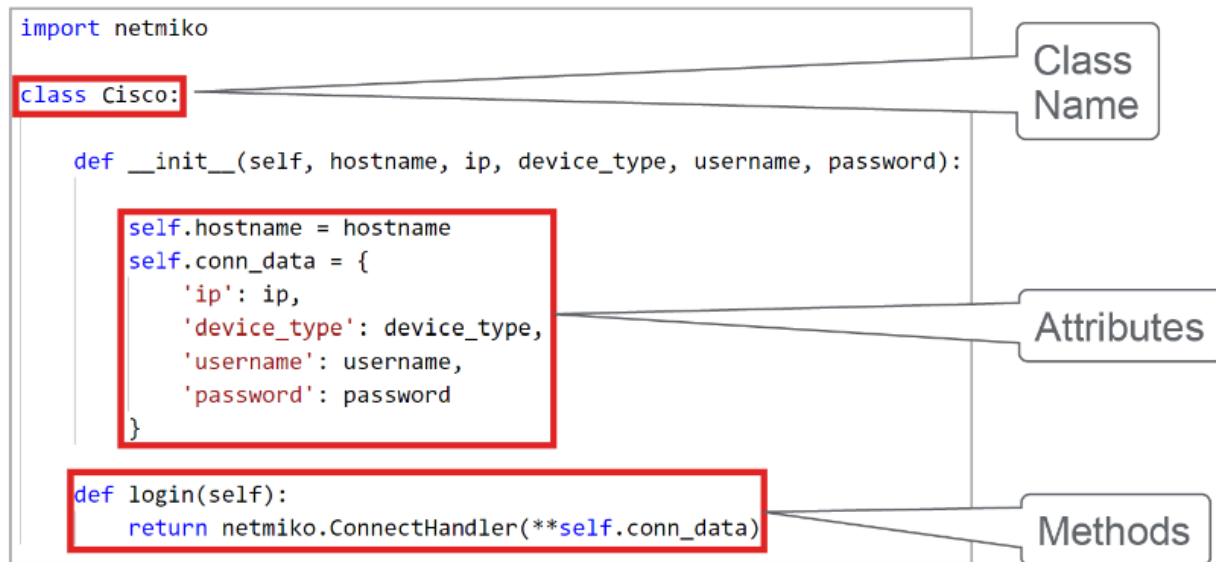
Method: sh ip int bri

Instance

Class: Switch1

Attributes: Hostname: Switch1
Management_IP: 192.168.5.2
VLAN_ID: 101
VLAN_Name: Client PC

Method: sh ip int bri



The `init` method is a reserved method that is also known as a constructor. Constructors are called when you want to create an object from the class. Arguments in the constructor are used as an object's variables.

There are two types of variables in a class; one is a class variable, and one is an instance variable. A class variable is assigned like any other variable and never changes, no matter how many instances you make. The only way to change this variable is to change your code. The command to make a class variable is as follows:

```
class Cisco:
    support = 'ciscosupport@cisco.com'
```

The `support = 'ciscosupport@cisco.com'` is the class variable. To make variables that change with each instance, use the `__init__` method. All attributes under the `__init__` method will be set when each instance is run.

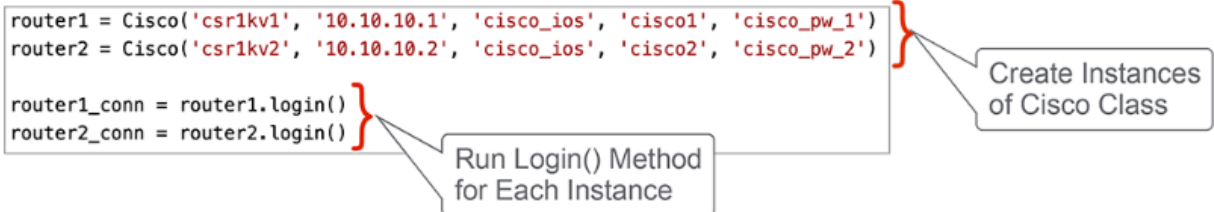
The first argument in the `__init__` method is `self`; this keyword can be anything you want it to be, but `self` is recommended. Self represents the instance of a class and allows the class to keep the different instances and the arguments that go with them separate. The code to create instance variables is as follows:

```
class Cisco:
    def __init__(self, hostname, ip, device_type, username, password):
        self.hostname = hostname
        self.conn_data = {
            'ip': ip,
            'device_type': device_type,
            'username': username,
            'password': password
        }

    def login(self):
        return netmiko.ConnectHandler(**self.conn_data)
```

Here, the Cisco class has the variables hostname, ip, device_type, username, and password.

The method login takes the self.conn_data information and unpacks it. It then uses netmiko to make an SSH connection to the device.



```
router1 = Cisco('csr1kv1', '10.10.10.1', 'cisco_ios', 'cisco1', 'cisco_pw_1')
router2 = Cisco('csr1kv2', '10.10.10.2', 'cisco_ios', 'cisco2', 'cisco_pw_2')

router1_conn = router1.login()
router2_conn = router2.login()
```

Create Instances of Cisco Class

Run Login() Method for Each Instance

Above, two instances of the class Cisco are run

In the code example above, you create two instances of the Cisco class, one called router1, and another called router2. When you create each instance through the process known as instantiation, you provide parameters that will be used for the connection. Each instance can have unique values for its attributes, but they will all share the same methods. After creating two instances, you call the login() method on each instance, which will return a connected SSH object to the variables router1_conn and router2_conn.

Using Inheritance to Extend Functionality

inheritance allows you to define a class that inherits all the methods and attributes from another class. Inheritance is important because it allows for code reusability and it represents real-world relationships. The class that is being inherited from is known as the base class or parent class. The class that inherits from another class is called the derived class or child class.

```

import netmiko

class CiscoIOSBase:
    """Base connectivity functions for Cisco IOS devices"""

    def __init__(self, ip, username=None, password=None):
        self.conn_data = {
            'ip': ip,
            'username': username,
            'password': password,
            'device_type': 'cisco_ios'
        }

    def login(self):
        self.conn = netmiko.ConnectHandler(**self.conn_data)

```

This parent class will act as a base class for connecting to Cisco IOS devices.

The children classes could specifically deal with IOS routers and switches.

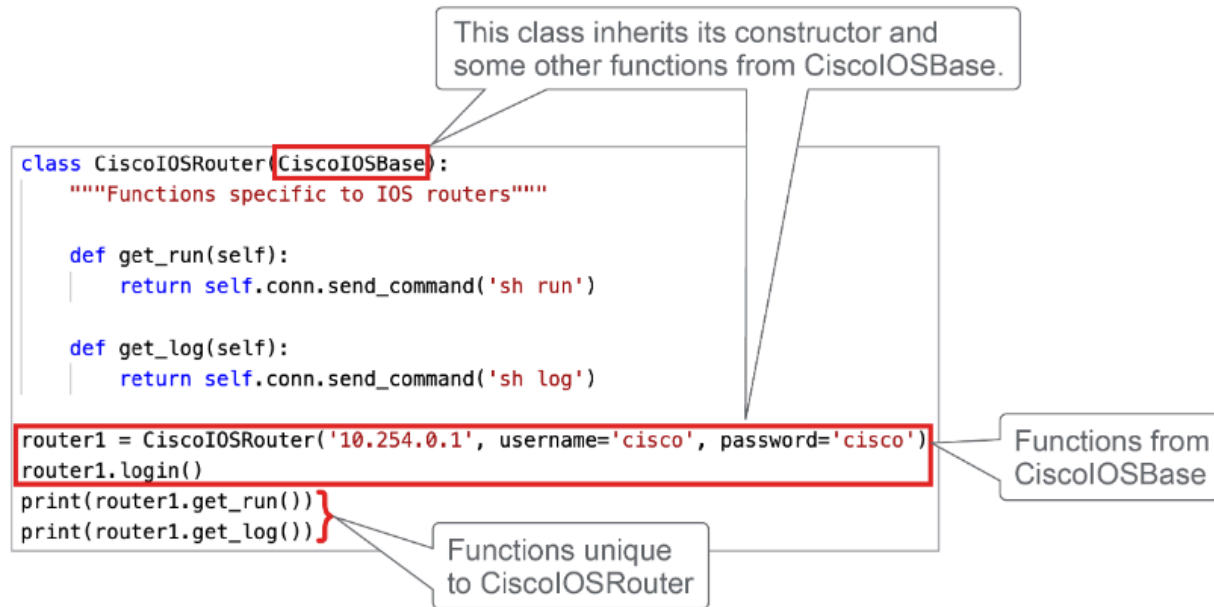
Deriving one class from another is part of code reuse; you get all the parent's attributes and methods without rewriting it. This makes your code cleaner, and it allows for faster development. In the following code, you are making a child class called CiscoIOSRouter from the parent class CiscoIOSBase.

```

class CiscoIOSRouter(CiscoIOSBase):
    pass

```

While the code is simple to explain the concept, it doesn't accomplish much because each class would be the same. Ideally, you want to inherit all the attributes and methods from the base class and add functionality to it.



The examples shown in the figure are the child class created from the parent class CiscoIOSBase. The child class CiscoIOSRouter has two new methods, `get_run()` and `get_log()`. Let's examine each of the lines of code:

- This command makes a new class inherited from the cisco class.

```
class CiscoIOSRouter(CiscoIOSBase):
```

- Because the `__init__()` function is not defined in the child class, it is inherited from the parent class. The child class also inherits the `login()` function and would inherit all other functions and attributes from the parent class.
- There are two methods defined in the CiscoIOSRouter class. The first method shown below will return the output from the command `sh run`.

```
def get_run(self):
    return self.conn.send_command('sh run')
```

The second method will return the output from the command `sh log`.

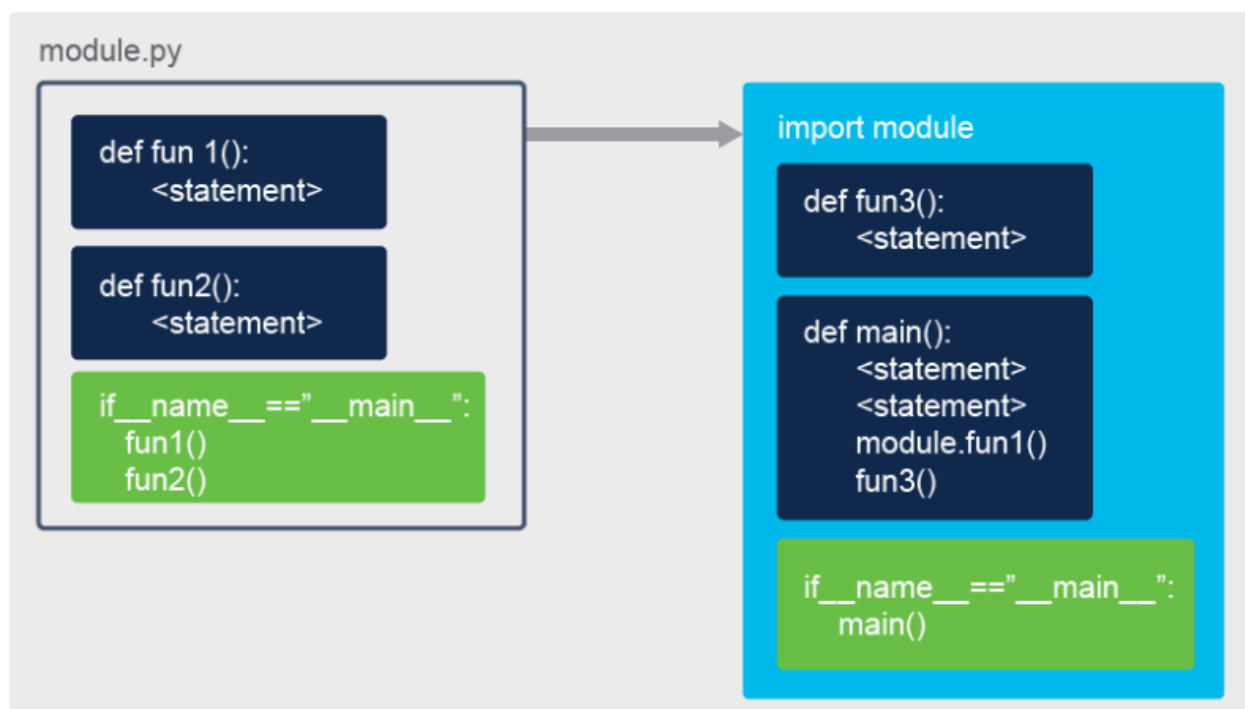
```
def get_log(self):
    return self.conn.send_command('sh log')
```

In the bounding box toward the bottom of the figure above, you create an instance of the child class `CiscoIOSRouter` using the constructor from the parent class, then `login()` to a device with another function inherited from the parent class. After you log in, you use the methods in the `CiscoIOSRouter` class to print the running config and log to the terminal.

To continue developing this class, you would most likely create functions in the class `CiscoIOSRouter` that are unique to routers. You could also create another class called `CiscoIOSSwitch` that inherits from `CiscoIOSBase` and would contain functions specific to switches.

Main Construct

Some programming languages have a special function called "main" that defines when a program starts to execute. This defining of the starting point is helpful to understand how a program works. The python interpreter runs each line of code from the top down without any special function to define the starting point. Because of this, programmers have come up with other ways to define the execution point. One of these ways is by using a **main()** function and the `__name__` property.



The `__name__` variable is a special built-in variable that shows the name of the current module. The name of the current module can change based on how the script was run. Looking at the example shown, if you run the script on the left from a python file, the

internal variable `__name__` is assigned the value `"__main__"`. However, if you import a module and it runs, the `__name__` of that module will be the module name. Therefore, Python's interpreter knows if the file is the main script that is running or an imported module. This is checked each time a python file is run.

Understanding the internal variable name allows you to build scripts that can run as the main file or be imported to another Python file, and only part of the file be executed. For example, the script on the left represents code that is made up of two functions (`fun1()`, `fun2()`), and an `if __name__ == "__main__"` statement that calls the two functions. If you execute this Python file, the internal variable `__name__` will be assigned `"__main__"`. When the interpreter runs the file from top-down, the `if __name__ == "__main__"` statement will evaluate to `True`, and the functions `fun1()` and `fun2()` will be called. However, if the code is imported to another Python file, then the internal variable `__name__` will equal the `module_name`. The Python file that imported it will have access to the two functions but nothing inside the `if` statement. This behavior allows you to control what runs based on if the script is imported by another Python file or not.

Now that you understand the `main()` function and the internal variable `__name__`, it is best practice to write your code to define all the functions you have written at the top of your program below the import statement. The last function should be the `main()` function. The `main()` function will contain the code in the order that you want it executed. In the example above, the code in the `main()` function will run the two statements and then `fun1()` and `fun2()`. After the `main()` function, you will have the `if __name__ == "__main__"` statement. Inside the `if` block, the function `main()` is called. Writing your code in this layout will make it easier for yourself and other developers to understand the execution point and flow of your code easier.

Data Storage Options

Data Formats

CSV files

ex.

```
1  Hostname, Vendor, Model, Location
2  sw1, Cisco, 3850, New York
3  sw2, Cisco, 3750, Chicago
4  sw3, Cisco, 3650, RTP
5  sw4, Cisco, 3850, San Jose
```

XML Data Format

```
1  <gigabitethernet>
2    <name>1</name>
3    <mop>
4      <enabled>false</enabled>
5      <sysid>false</sysid>
6    </mop>
7    <negotiation xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-ethernet">
8      <auto>true</auto>
9    </negotiation>
10 </gigabitethernet>
```

To access and modify XML documents, the World Wide Web Consortium developed an application programming interface (API) known as Document Object Module (DOM). DOM represents an XML document in a tree structure. This structure is broken down into a root element that acts as the parent of all other elements in XML, child elements, and sub-child elements. An element is a container to hold text, other elements, and attributes.

The following is an example of XML elements and their definitions:

```

<router>
  <interface>
    <gi0/1>
      <name>gi0/1</name>
      <ip>192.168.5.1</ip>
    </gi0/1>
    <gi0/2>
      <name>gi0/2</name>
      <ip>192.168.5.2</ip>
    </gi0/2>
  </interface>
</router>

```

Root: There is only one root in a document. It is the first node and it encloses all other elements. The root is also known as a document element. In the example, the root element is <router>.

Child: A child element belongs to only one parent. There can be multiple child elements under each parent. In the example, the child elements for <interface> are <gi0/1> and <gi0/2>. The elements of <name> and <ip> are also child elements with <gi0/1> and <gi0/2> are the parents.

Sub-Child: Elements <name> and <ip> are sub-children of <interface>.

Data: Data is the text between elements. In the example, “gi0/1” or “192.168.5.1” is data.

Comments: Comments are text in an XML document that are not processed. Comments are used to provide information about the XML document. The format of an XML comment is <!--This is an XML doc for router interfaces-->.

XML elements are known as tags. Tags are used to identify, store, and organize data. Each element has an opening and closing tag. The tags are case-sensitive and must match. The following are examples of correct and incorrect tagging:

- Correct Tag: <Name> Bill</Name>
- Incorrect Tags: <Name>Bill</name>

Whitespace for XML may or may not be significant. Whitespaces at the beginning of a tag are optional and can be used for readability purposes, but whitespaces inside tags (<Router IP>) and in between tags <Name> Bill Jones</Name> are essential.

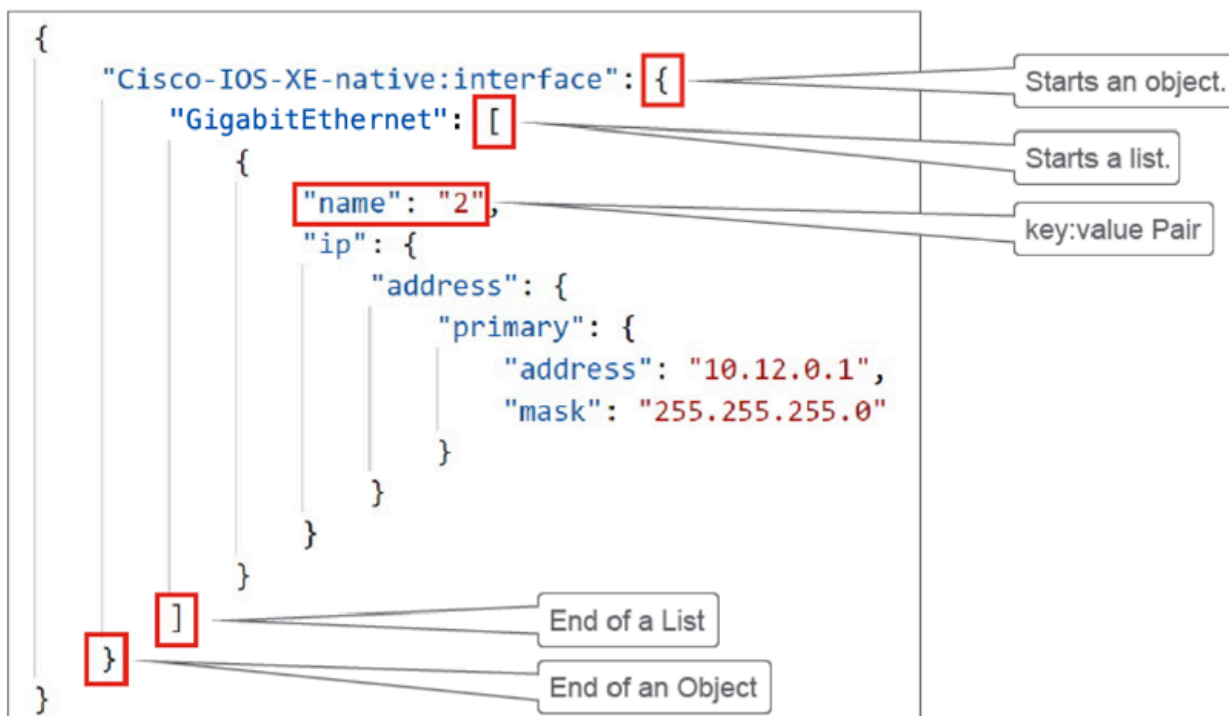
XML data is used in multiple situations:

- Exchanging information between systems
- Storing configuration files
- Populating log files

In networking, XML is used as a data format to exchange information. You will see XML used as the data encoding for application programming interfaces (APIs), Network Configuration Protocol (NETCONF), and Representational State Transfer Configuration Protocol (RESTCONF). NETCONF uses XML to encode the protocol messages and configuration data that are modeled by Yet Another Next Generation (YANG).

JSON Data Format

JavaScript Object Notation (JSON) is based on a subset of the JavaScript Programming Language Standard ECMA-262 3rd Edition - December 1999. It is a lightweight data-interchange format that is easy for humans to read and write and easy for machines to parse and generate. These properties make JSON an ideal data-interchange language.



JSON is built on objects and arrays.

An object is a set of unordered key/value pairs. An object begins with a `{` left brace and ends with a `}` right brace. Each key in the object is followed by a `:` colon and then the value. The key/value pairs are separated by a `,` comma except for the last key/value pair. All keys are strings surrounded by double quotes `" "`. The value of the key can be strings surrounded by double quotes `" "`, integers, floating points, Null, Boolean, objects, or arrays.

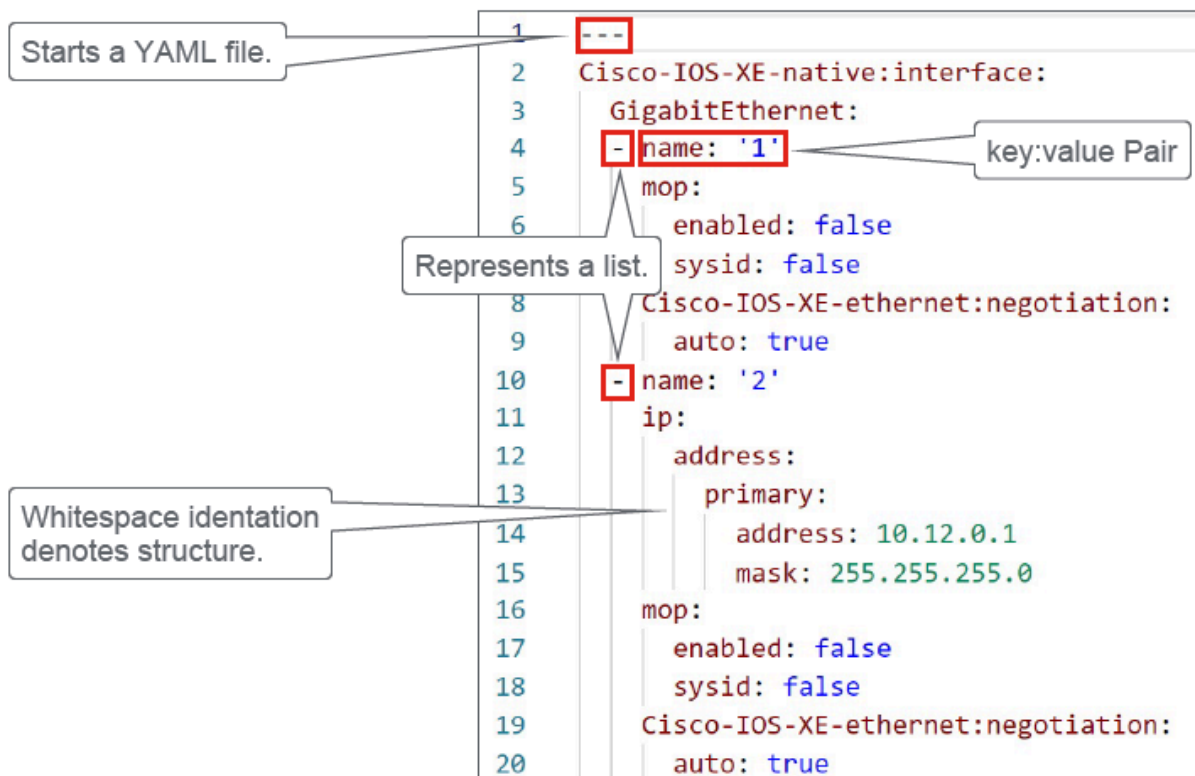
An array is a collection of ordered values that begins with a `[` left bracket and ends with a `]` right bracket. Values are separated by a `,` comma.

In JSON, whitespaces are insignificant and are only used to make the data easier for humans to read. The extension of a JSON file is .json.

JSON format and Python dictionaries are similar in that they both have key:value pairs. This similarity makes it easy to convert from one to the other.

YAML Data Format

YAML (YAML Ain't Markup Language) is a data serialization language, commonly used for configuration files. YAML files can have an extension of .yaml (preferred) or .yml.



Every YAML document begins with (---) three dashes.

Most of a YAML document is made up of key/value pairs. The bounding box on line 4 of the example shows a pair where the key is name and the value is 1. Note that with YAML the keys do not have quotes around them and there are no delimiters at the end of each line.

YAML relies on indentation for structure and the best practice is to use two whitespaces for each indentation. The use of tabs is not allowed.

Each new item in a list begins with a (–) hyphen and space. You can add comments to the file with the (#) pound symbol.

The following is an example of how (#) and (–) are used in a list:

```
# List of device names
Routers:
-   R1
-   R2
Switches:
-   SW1
-   SW2
```

Databases

Relational

- MySQL
- PostgreSQL
- Oracle

Nonrelational

- MongoDB (Document-Oriented Database)
- InfluxDB (Time Series Database)
- Graph Engine (Graph Database)

Reading from a Simple Text and CSV File

2 steps:

open file you want to work with

choosing the read method: **read()**, **readline()** or **readlines()**

Text File

```
Router.txt
1 Router1 is located in San Jose
2 Router2 is located in Chicago
3 Router3 is located in RTP
```

Code to Read .txt File

```
1 with open("router.txt", "r") as f:
2     content = f.read()
3
4     print(content)
```

Output

```
C:\PRNE>python readtxt.py
Router1 is located in San Jose
Router2 is located in Chicago
Router3 is located in RTP
```

open methods

open(filename, mode)

with open("filename", mode) as f

modes:

1. r - Opens a file for read-only, returns an error if the file does not exist.
 2. a - Opens a file appending, creates the file if it does not exist.
 3. w - Opens a file for write, creates the file if it does not exist.
 4. x - Creates the specified file, returns an error if the file exists.
- f: The variable that the file object will be referred to as.

The **with open** command is the easier method to use because you do not have to worry about closing the file when you are done; the **with open** command takes care of closing the file for you. This behavior is important so that files are not left open, preventing other users from accessing them.

To read the contents of a file, the **read()** method returns the contents of the file as individual strings, making it easy to manipulate the data inside the file.

The example is broken into three sections: the top section is the contents of a file named Router.txt, the middle section is the code used to open the file to read it, and the last section is the output of the code.

This line of code opens a file named Router.txt for read-only and saves the file object in the variable "f." In the example, the variable "f" is used, but any variable name can be used.

The file object "f" is then handed to the **read()** method. The command **content = f.read()** returns the bytes in a form of a string and saves it in a variable "content." The code then prints the returned bytes stored in content with the command **print(content)**.

With no parameter specified in the **read()** method, the complete file is read. If you only want some bytes read, you can specify it with the command **f.read(n)**, where "n" specifies the number of bytes to read. Therefore, if you only wanted to print the word Router1, you could use the command **print(f.read(7))**, where "7" represents the number of bytes to print.

The second method that you can use to read data in a file is the **readline()** method. This method returns each line one at a time. Therefore, if you used the command **print(f.readline())** in the example, the output would be the first line in the file, "Router1 is located in San Jose."

The third method is **readlines()**. The **readlines()** method returns a list of all the lines. Since the returned value is a list, you can control which lines you want by splicing. For example, to print the second line in the file, you would use the command **print(f.readlines()[1:2])**. The output would be ['Router2 is located in Chicago\n'].

Reading from a CSV File

CSV File

```
1  Hostname, Vendor, Model, Location
2  sw1, Cisco, 3850, New York
3  sw2, Cisco, 3750, Chicago
4  sw3, Cisco, 3650, RTP
5  sw4, Cisco, 3850, San Jose
```

Code to Read csv File

```
1  import csv
2
3  with open('devices.csv', 'r') as f:
4      devices = csv.reader(f)
5      header = next(devices)
6      print(f'Headers: {header}')
7      for row in devices:
8          print(row)
```

Output

```
C:\PRNE>python read_csv.py
Headers: ['Hostname', 'Vendor', 'Model', 'Location']
['sw1', 'Cisco', '3850', 'New York']
['sw2', 'Cisco', '3750', 'Chicago']
['sw3', 'Cisco', '3650', 'RTP']
['sw4', 'Cisco', '3850', 'San Jose']
```

The example has three areas. The first area shows the data of a CSV file named `devices.csv`. This file contains a list of devices with each device's hostname, vendor, model, and location.

The second area of the example is the script that reads the CSV file. A breakdown of the script follows.

To read a CSV file, you use a built-in Python standard library called `csv`. To use the library, you need to import it with the command **`import csv`**. Notice that the library name is all lowercase `csv`, not uppercase `CSV`.

The code opens a file named `devices.csv` for read-only and saves the file object in the variable “`f`.” In the example, the variable “`f`” was used, but any variable name can be used.

The file object “`f`” is then handed to **`csv.reader()`**. The **`csv.reader()`** iterates over the file and returns the data line by line. In this example, the code stores the iteration in a variable named `device`.

Next, headers and then the information in each row are printed. To get the header information, use the command **`header = next(devices)`**. This command reads the first line iterated by **`csv.reader()`** and stores it in the variable `header`. The **`next()`** method moves the iterator to the next line and waits for a command.

Finally, the code uses a for loop to iterate the file with the command **`for row in devices:`** and prints each line in the file using the command **`print(row)`**.

If you don't want to work with a list and prefer to work with a dictionary, you can use the **`csv.DictReader()`** method, as in the second example. This works similarly to **`csv.reader()`**, but instead of saving each row as a list, each will be saved as a dictionary with headers as keys.

CSV File

```
1  Hostname, Vendor, Model, Location
2  sw1, Cisco, 3850, New York
3  sw2, Cisco, 3750, Chicago
4  sw3, Cisco, 3650, RTP
5  sw4, Cisco, 3850, San Jose
```

Code to Read csv File

```
1  import csv
2
3  with open('devices.csv', 'r') as f:
4      devices = csv.DictReader(f)
5      for row in devices:
6          print(row)
7          print(row['Hostname'], row['Model'])
8          print('-----')
```

Output

```
C:\PRNE>python read_csv.py
{'Hostname': 'sw1', 'Vendor': 'Cisco', 'Model': '3850', 'Location': 'New York'}
sw1 3850
-----
{'Hostname': 'sw2', 'Vendor': 'Cisco', 'Model': '3750', 'Location': 'Chicago'}
sw2 3750
-----
{'Hostname': 'sw3', 'Vendor': 'Cisco', 'Model': '3650', 'Location': 'RTP'}
sw3 3650
-----
{'Hostname': 'sw4', 'Vendor': 'Cisco', 'Model': '3850', 'Location': 'San Jose'}
sw4 3850
-----
```

The middle section of the example shows how to use the `csv.DictReader()` method. The command takes the contents stored in the variable “f” and saves it as a dictionary stored in the variable “devices.”

```
devices = csv.DictReader(f)
```

You can then use a for loop to iterate through each row in devices and print the dictionary.

Writing Data to a Simple Text and CSV File

To write data to a .txt file, you need to determine if you are writing to a file or appending to a file and which file writing method you want to use, either the `file.write()` method or the `file.writelines()` method.

Writing Data to a .txt File

```
1 data = "Switch1 is located in San Jose"
2
3 with open("switch.txt", "w") as f:
4     f.write(data)
```

Contents of switch.txt File

```
switch.txt
1 Switch1 is located in San Jose
```

```
with open("switch.txt", "w") as f:
```

This command opens a file named `switch.txt` in the mode “w.” The “w” mode opens a file for writing and, if the file does not exist, it will create the file. If the file exists, its contents will be deleted and you will write to an empty file. The file is opened and stored in the variable “f.”

Once the file is open, you can use the `write()` method to write data to the file with the following command:

```
f.write(data)
```

The variable “f” represents the file the data will be written to and the parameter “data” is the data to be written to the file. In this example, data is a variable holding the string “Switch1 is located in San Jose.”

The result of the script, shown in the example, is that the string “Switch1 is located in San Jose” is written to the file `switch.txt`.

Note:

You can also use the **print()** function to write data to a file using the keyword argument **file**. In many scenarios, **print()** is more powerful than **file.write()** because of the additional functionality built into **print()**. The **print()** function enables you to use any character or characters that you want to separate the values you are printing with the keyword argument **sep**. The keyword argument **end** will add a newline character ("**\n**") by default after the last value you print, but you can change that too.

Appending Data

To add data to an existing file, use append mode instead of write mode.

Appending Data to a .txt File

```
1 data = "Switch2 is located in Chicago"
2
3 with open('switch.txt', 'a') as f:
4     f.write(data)
```

Updated switch.txt File

```
switch.txt
1 Switch1 is located in San JoseSwitch2 is located in Chicago
```

Use append mode in the **with open** command by using the parameter "**a**":

```
with open("switch.txt", "a") as f:
```

The parameter "**a**" represents append mode.

Once you open the file, you can use the **write()** method to write data to the file. As shown in the example, the data is written to the end of the last line in the file. Since the file **switch.txt** has already been created with the string "Switch1 is located in San Jose," the new data is added to the end of the last line in the file and the result is "Switch1 is located in San JoseSwitch2 is located in Chicago."

To solve the problem of adding the data to the last line in the file, you can include a command to create a new line before you add the new data.

Appending Data to a .txt File on a New Line

```
1 data = "Switch2 is located in Chicago"
2
3 ✓ with open('switch.txt', 'a') as f:
4     f.write('\n')
5     f.write(data)
```

Updated switch.txt File

```
1 Switch1 is located in San Jose
2 Switch2 is located in Chicago
```

To append data on a new line of an existing file, add the following command to the script:

```
f.write('\n')
```

Writing to a List of Data

If you want to write a list of data to a file, you can use the method `file.writelines()`.

Writing Data to a .txt File with writelines()

```
1 data = ["Switch3 is located in Miami", "\n" "Switch4 is located in London"]
2
3 ✓ with open('switch.txt', 'w') as f:
4     f.writelines(data)
```

switch.txt File

```
≡ switch.txt
1 Switch3 is located in Miami
2 Switch4 is located in London
```

o write to a list, open the file with the **with open** command and then use the command:

```
f.writelines(data)
```

The parameter “data” is a list holding two strings. Note the newline escape character (“\n”) between the two strings in the list data. Without this escape character, the two strings would be written to the file as one line: "Switch3 is located in MiamiSwitch4 is located in London."

Since the new line escape character was used in the example, the two strings are printed on separate lines.

Writing Data to a CSV File

Code to Write to a CSV File

```
1  import csv
2
3  data = [['Hostname', 'Vendor', 'Model', 'Location'],
4          ['sw10', 'Cisco', '3800', 'Miami'],
5          ['sw11', 'Cisco', '3650', 'Atlanta']]
6
7  with open('newdevices.csv', 'w') as f:
8      devices = csv.writer(f, quoting=csv.QUOTE_NONNUMERIC)
9      for row in data:
10         devices.writerow(row)
11
12  with open('newdevices.csv', 'r') as f:
13      print(f.read())
```

Output

```
C:\PRNE>python write_csv2.py
"Hostname","Vendor","Model","Location"

"sw10","Cisco","3800","Miami"

"sw11","Cisco","3650","Atlanta"
```

The Python script in the example reads the information stored in the variable “data,” opens a new file named newdevices.csv, and writes the data to the file.

To open the file, use the command:

```
with open('newdevices.csv', 'w') as f:
```

The file name is "newdevices.csv" and “w” means you are writing to the file.

The **csv.writer()** method takes the user data and turns each row into a delimited string that can be used to make a CSV file.

```
devices = csv.writer(f, quoting=csv.QUOTE_NONNUMERIC)
```

The variable “f” is the file object of newdevices.csv and **quoting** tells the writer what you want quotes. **QUOTE_NONNUMERIC** means to put quotes around everything except numeric values.

To iterate over each delimited string of data and write it to a csv file, use the code:

```
for row in data:  
    devices.writerow(row)
```

Note

Although not shown, another way to write data to a file is with the **print()** statement. The print statement allows for more functionality such as formatting. To use **print()** to write to a file, use the command **print(data, file = newdevices.csv)**.

In the example, the for loop takes each row in the variable data and writes the delimited data to the newdevices.csv file. To view the output, the script takes the newdevices.csv file and prints it.

Reading Data from a JSON Text File

While it is useful to understand how to read and write simple text files, most APIs today send their data in a JSON format. Therefore, it is important to understand how to read JSON data and convert it into an object that you can work with in Python.

This topic will explain how to write a Python script to read data from the JSON file `csr_interface_IPs.json`.

`csr_interface_IPs.json` file (partial)

```
{  
  "csr_interface_IPs.json" > ...  
  1  {  
  2    "Cisco-IOS-XE-native:interface": {  
  3      "GigabitEthernet": [  
  4        {  
  5          "name": "2",  
  6          "ip": {  
  7            "address": {  
  8              "primary": {  
  9                "address": "10.12.0.1",  
10                "mask": "255.255.255.0"  
11              }  
12            }  
13          },  
14          "mop": {  
15            "enabled": false,  
16            "sysid": false  
17          },  
18          "Cisco-IOS-XE-ethernet:negotiation": {  
19            "auto": true  
20          }  
21        },  
      ]  
    }  
  }
```

To read data from a JSON file, you use a built-in Python standard library named `json`. The `json` library allows you to encode and decode JSON data. To import this library, issue the following command:

```
import json
```

Code to Read Data from a JSON File

```
1 import json
2 from pprint import pprint
3
4 with open ('csr_interface_IPs.json', 'r') as f:
5     ip = json.load(f)
6
7 pprint(ip)
```

Output

```
C:\PRNE>python read_json.py
{'Cisco-IOS-XE-native:interface': {'GigabitEthernet': [{'Cisco-IOS-XE-ethernet:negotiation': {'auto': True},
  'ip': {'address': {'primary': {'address': '10.12.0.1',
    'mask': '255.255.255.0'}}}},
  'mop': {'enabled': False,
    'sysid': False},
  'name': '2'},
  {'Cisco-IOS-XE-ethernet:negotiation': {'auto': True},
  'ip': {'address': {'primary': {'address': '10.13.0.1',
    'mask': '255.255.255.0'}}}},
  'mop': {'enabled': False,
    'sysid': False},
  'name': '3'},
  {'Cisco-IOS-XE-ethernet:negotiation': {'auto': True},
  'description': 'MANAGEMENT_NETWORK ',
  'do_not_change': 'DO_NOT_CHANGE',
  'ip': {'address': {'primary': {'address': '10.254.0.1',
    'mask': '255.255.255.0'}}}},
  'mop': {'enabled': False,
    'sysid': False},
  'name': '4'}}]}
```

The `json.load()` method translates JSON data to Python objects in the following way:

JSON	Python
Object	Dictionary
Array	List, tuple
String	str
number	Int, float
true	True
false	False
null	None

Note: There is also a json method called **loads()**, with an “s” at the end of load. **json.loads()** does not decode a JSON file to a Python object like **json.load()**, but instead decodes a JSON string to a Python object.

In the example, the script output shows the results of printing the Python object stored in the variable “ip.” To make the output easy to read, the script uses a module called **pprint()**.

The **pprint()** module provides an alternative to Python’s standard print capability. The **pprint()** module prints data structures in a well-formatted way that is more readable than using the **print()** function. Using **pprint()** with no parameters sets default formats, but you can configure specific formatting with parameters as shown below:

```
pprint(object, indent=1, width=80, depth=None)
```

- object: Object to print.
- indent: Number of characters to indent each line.
- width: Maximum number of characters in each line.

- depth: controls how far down into the nested data pprint will look.

To use the `pprint()` module, you must first import it with the command `from pprint import pprint`.

Once you import the module, you can use its method with the command `pprint(object)`.

Based on the data structure of the object and the parameters that you choose, you determine how the output will look. To learn more about the pprint module visit <https://docs.python.org/3/library/pprint.html>.

Writing Data to a JSON Text File

Code

```
write_json.py
1 import json
2 from pprint import pprint
3
4 json_data = [
5     {
6         'name': 'Router1',
7         'ip': '192.168.5.5',
8         'mask': '255.255.255.0'
9     }, {
10        'name': 'Router2',
11        'ip': '192.168.10.10',
12        'mask': '255.255.255.0'
13    }
14 ]
15 with open('newjsonfile.json', 'w') as f:
16     json.dump(json_data, f, indent=2)
17
18 with open('newjsonfile.json', 'r') as f:
19     pprint(json.load(f), width=40)
```

Output

```
[{'ip': '192.168.5.5',
  'mask': '255.255.255.0',
  'name': 'Router1'},
 {'ip': '192.168.10.10',
  'mask': '255.255.255.0',
  'name': 'Router2'}]
```

Method to write Python data to file as formatted JSON.

The function `json.load(file)` will interpret the contents of a JSON file as python data.

Data Access in Raw or Unstructured Format

There may be times when you need to manipulate or write unstructured data to a file. For example, when you use the Netmiko library to make a Secure Shell (SSH) connection to a device and execute a show command, the returned data is in an unstructured format. There are a couple of ways that you can handle this unstructured data. One is to use **regular expressions to find the information you want and save it to a dictionary**, and the other is to use **TextFSM with Netmiko to have the data saved in a JSON format**.

Using Regular Expressions to save to a dictionary

Using re module

```
1 import netmiko
2 import re
3 from pprint import pprint
4
5 conn = netmiko.ConnectHandler('10.254.0.1', username='cisco',
6                               password='cisco', device_type='cisco_ios')
7 sh_ip_int_br = conn.send_command('show ip int brief')
8
9 interface_pattern = r'(GigabitEthernet[1-9])\s'
10 status_pattern = r'GigabitEthernet[1-9]\s.*?(up|administratively down|down)'
11
12 interface_names = re.findall(interface_pattern, sh_ip_int_br)
13 interface_statuses = re.findall(status_pattern, sh_ip_int_br)
14 interface_dict = dict(zip(interface_names, interface_statuses))
15
16 print(interface_names)
17 print(interface_statuses)
18 pprint(interface_dict)
```

Import re module.

Using re.findall() to match patterns and return lists.

Combining two lists into a dictionary.

Output

```
['GigabitEthernet1', 'GigabitEthernet2', 'GigabitEthernet3', 'GigabitEthernet4']
['administratively down', 'up', 'up', 'up']
{'GigabitEthernet1': 'administratively down',
 'GigabitEthernet2': 'up',
 'GigabitEthernet3': 'up',
 'GigabitEthernet4': 'up'}
```

To match the interface name, the pattern `r'(GigabitEthernet[1-9])\s'` is used and stored in the variable `interface_pattern`. This pattern will match anything that starts with GigabitEthernet and has a digit 1-9 at the end, followed by a space. The capturing group does not include the whitespace, so you will not see whitespace in the list. This whitespace ensures that you will only match on GigabitEthernet1-9 and not any GigabitEthernet interfaces with a double-digit number.

To match the status of each interface, use the pattern `r'GigabitEthernet[1-9]\s.*?(up|administratively down|down)'`. This pattern must use "up," "administratively down," and "down" as the three interface status options, and the vertical line separating them means "or." Therefore, the script looks for any of those three words.

The patterns are then used in a regular expression method that finds all the matches in the output of the `sh ip int bri` command. This example uses the `findall()` method with the following code:

```
interface_names = re.findall(interface_pattern, sh_ip_int_br)
```

```
int_statuses = re.findall(status_pattern, sh_ip_int_br)
```

The only difference between the two commands is the pattern that is used. The `interface_names` variable is storing a list of all matches of the pattern `interface_pattern`. Here is the result:

```
['GigabitEthernet1', 'GigabitEthernet2', 'GigabitEthernet3',  
'GigabitEthernet4']
```

The **int_statuses** variable is storing a list of all matches of the pattern **status_pattern**. Here is the result:

```
['administratively down', 'up', 'up', 'up']
```

Notice that the **interface_names** list has four interfaces and the **interface_statuses** list has four statuses, because the information is matched in the status column, but only when the pattern GigabitEthernet[1-9]\s occurred earlier on that line.

At this point in the coding, you have two lists that you can use in your program. But in the example code, you can take it a step further and make a dictionary where the key is the interface name and the value is the status. To take two lists and make a dictionary, use the function **zip** along with the function **dict()**. The **zip** function takes the two lists and returns a list of tuples, and the **dict()** function takes the list of tuples and returns a dictionary. The following command uses these two functions:

```
interface_dict = dict(zip(interface_names, int_statuses))
```

In this command, the inside function is **zip(interface_name, int_status)**. This function takes the **interface_names** and the **interface_statuses** lists and combines them into a list of tuples, each tuple containing the items at index value N from each list. The code then takes the data returned from the **zip()** function and uses it as the argument for the **dict()** function. This process takes the list of tuples and turns them into a dictionary. Here is the result:

```
{'GigabitEthernet1': 'administratively down', 'GigabitEthernet2':  
'up', 'GigabitEthernet3': 'up', 'GigabitEthernet4': 'up'}
```

Although not shown, if you want to take the dictionary and write it to a file in a JSON format, you can use the command **json.dump(interface_dict)**.

Using TextFSM with Netmiko

```

1  import netmiko
2  import json
3
4  csr = netmiko.ConnectHandler('10.254.0.1', username='cisco',
5  | password='cisco', device_type='cisco_ios')
6  sh_ip_int_br = csr.send_command('show ip int brief', use_textfsm=True)
7
8  with open('sh_ip_int_br.json', 'w') as f:
9  | json.dump(sh_ip_int_br, f, indent=2)

```

Formats text as a list of dictionaries.

```

1  import netmiko
2  import json
3
4  csr = netmiko.ConnectHandler('10.254.0.1', username='cisco',
5  | password='cisco', device_type='cisco_ios')
6  sh_ip_int_br = csr.send_command('show ip int brief', use_textfsm=True)
7
8  with open('sh_ip_int_br.json', 'w') as f:
9  | json.dump(sh_ip_int_br, f, indent=2)

```

sh_ip_int_br.json

```

[
  {
    "intf": "GigabitEthernet1",
    "ipaddr": "unassigned",
    "status": "administratively down",
    "proto": "down"
  },
  {
    "intf": "GigabitEthernet2",
    "ipaddr": "10.12.0.1",
    "status": "up",
    "proto": "up"
  },
  {
    "intf": "GigabitEthernet3",
    "ipaddr": "10.13.0.1",
    "status": "up",
    "proto": "up"
  },
  {
    "intf": "GigabitEthernet4",
    "ipaddr": "10.254.0.1",
    "status": "up",
    "proto": "up"
  }
]

```

Creates file called sh_ip_int_br.json
and writes formatted json data to file

The parameter **use_textfsm=True** instructs Netmiko to take the unstructured data that is being returned and format it in a list of dictionaries.

The TextFSM module was developed by Google and uses templates to parse any text-based CLI output. TextFSM was implemented in Netmiko version 2.0. **TextFSM templates contain variables and rules with regular expressions.** A template file defines how to parse the output into a structured format. You can build your own TextFSM templates, or you can find various templates on GitHub.

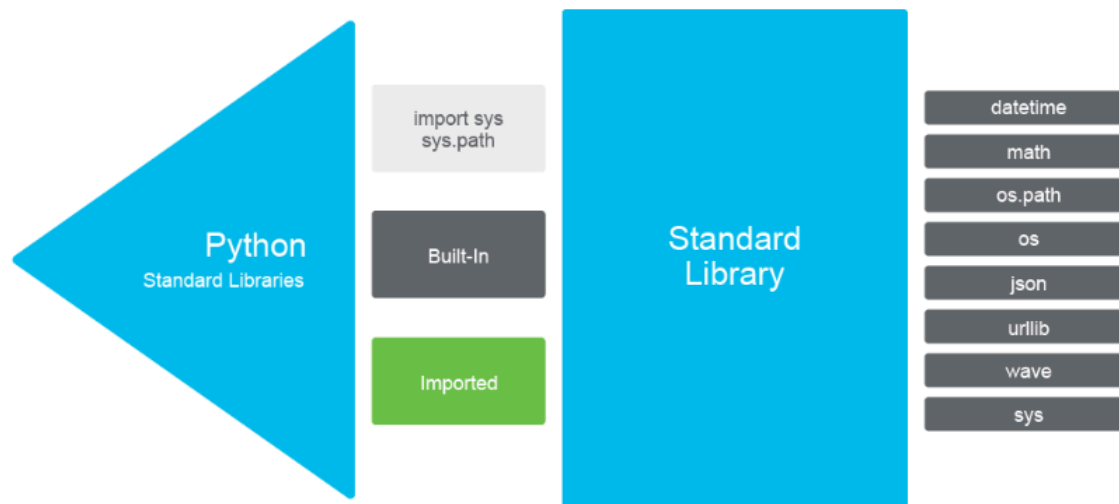
A TextFSM template has two parts—Value and Start:

- Value: Defines the fields that should be extracted. INTF, IPADDR, STATUS, and PROTO are variables that will be used in the regular expression. Following each variable is a description of what is the expected output for that variable.
- Start: Signifies the beginning of the template that defines the rules to parse the content. The rule is a regular expression using the previously mentioned variable. The term “record” at the end of the command means that each line that matches the regular expression will be processed.

For more information on Netmiko and TextFSM, you can visit <https://pynet.twb-tech.com/blog/automation/netmiko-textfsm.html>.

Python Modules and Packages

Python Standard Library Modules



To see a list of all libraries available to use, use the following command.

```
>>> help("modules")
```

To determine which directory that Python checks for the standard library, run the following command in the Python shell:

```
>>> import sys
```

```
>>> sys.path
```

The modules of the standard library include the following:

- **Date and Time:** Supplies classes for output formatting and manipulating dates and times.
 - `datetime`: Allows for manipulation of date and times
 - `calendar`: Provides useful functions related to a calendar
- **Mathematical:** Provides numeric and math-related functions and data types.
 - `math`: Mathematical functions
 - `Random`: Generates pseudo-random numbers
- **File System:** Provides an interface to files and file structure on the operating system.
 - `os.path`: Allows for pathname manipulation
 - `filecmp`: Compares files and functions
- **Operating System:** Provides interfaces to operating system features available on almost all operating systems, such as io, logging, and a clock.
 - `os`: Allows you to use operating system-dependent functionality
 - `logging`: Logging system for applications and libraries
- **Read/Write Data Formats:** Reads and writes to data formats such as XML and JSON.
 - `json`: Reads and writes json data
 - `base64`: Encodes and decodes binary data
- **Internet Protocol:** Supports Internet protocols and related technology such as FTP, HTTP, and SMTP.
 - `urllib`: Opens, reads, parses URLs
 - `ftplib`: Implements client-side FTP protocol
- **Multimedia Data:** Provides various algorithms or interfaces that are mainly useful for multimedia applications such as sound and video.
 - `wave`: Interface to read and write WAV sounds
 - `imghdr`: Determines the type of image contained in a file
- **Run time:** Provides services related to the Python interpreter and its interaction with its environment.
 - `sys`: Provides access to some variables used by the interpreter
 - `traceback`: Extracts, formats, and prints stack traces

The above information and other related information can be found at [docs.Python.org/3/library/](https://docs.python.org/3/library/).

Reusable Code with Modules and Packages

finding code:

<https://pypi.org/>

<https://developer.cisco.com/codeexchange>

Unlike the standard library, packages from the greater Python community will need to be installed on your device to use them. To install a package, use the following CLI command:

```
python -m pip install "package name"
```

By default, when you install a third-party library, its package and all its dependencies are stored in a folder named site-packages. These libraries can be globally installed, meaning they are available to all users on a computer, or locally installed, meaning they will only be available to the user that performed the installation. You can find the location of site-packages by entering the Python interpreter and running the following commands:

```
>>> import site
>>> site.getsitepackages()
['c:\\Users\\username\\AppData\\Local\\Programs\\Python\\Python3
9\\lib\\site-packages']
```

The example gives you a list of packages installed. Keep in mind as you work on different projects that it is possible that you would need a different version of the same package. To work with a different version of the same package on different projects, you need to use virtual environments. Virtual environments are not covered in this course, but you can learn more about them at <https://docs.python.org/3/tutorial/venv.html>.

Command to Install
Requests Package
and its Dependencies

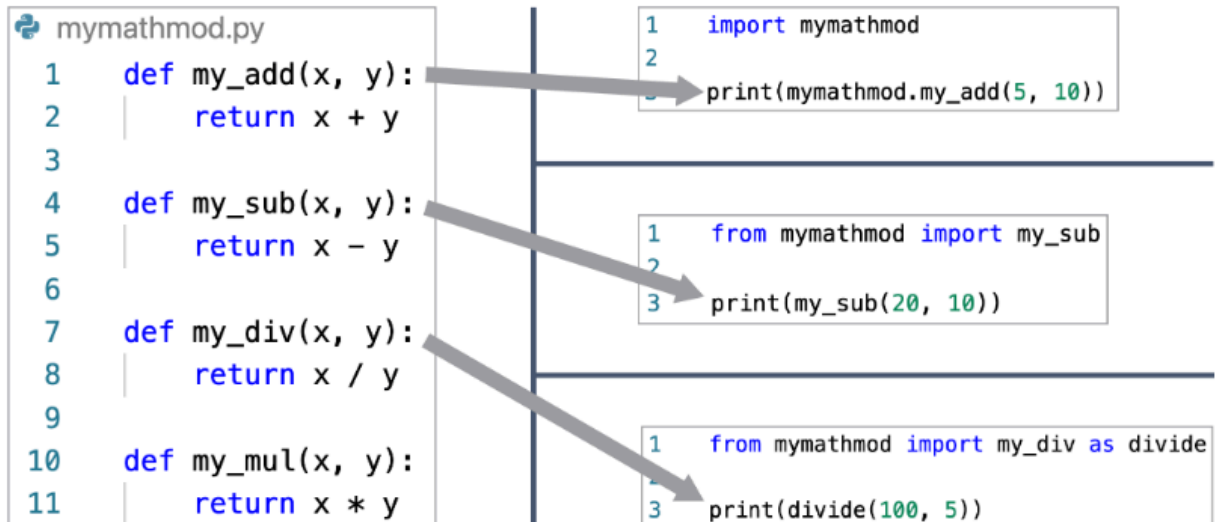
```
C:\PRNE> pip install requests
Collecting requests
  Downloading requests-2.25.1-py2.py3-none-any.whl (61 kB)
    | 61 kB 5.4 MB/s
Collecting idna<3,>=2.5
  Downloading idna-2.10-py2.py3-none-any.whl (58 kB)
    | 58 kB 5.5 MB/s
Collecting certifi>=2017.4.17
  Downloading certifi-2021.5.30-py2.py3-none-any.whl (145 kB)
    | 145 kB 2.6 MB/s
Collecting chardet<5,>=3.0.2
  Downloading chardet-4.0.0-py2.py3-none-any.whl (178 kB)
    | 178 kB 7.1 MB/s
Collecting urllib3<1.27,>=1.21.1
  Downloading urllib3-1.26.6-py2.py3-none-any.whl (138 kB)
    | 138 kB 24.3 MB/s
Installing collected packages: urllib3, idna, chardet, certifi, requests
Successfully installed certifi-2021.5.30 chardet-4.0.0 idna-2.10 requests-2.25
.1 urllib3-1.26.6
```

Package and
Dependency
Installation Status

```
C:\PRNE> pip list
Package      Version
-----
certifi      2021.5.30
chardet      4.0.0
idna         2.10
pip          21.1.3
requests     2.25.1
setuptools   49.2.1
urllib3      1.26.6
```

Installed Packages
and Versions now
Including Requests
and its Dependencies

Not only can you find third-party packages and modules that you can install and use, but also you can build your own modules or packages. A module is code that you can reuse and can be written in either Python or C. This code could include constants or functions that you can call into your script.



top example imports the whole module

2nd example imports just a specific function

3rd example imports a specific function and **renames** it
or even the module can be renamed:

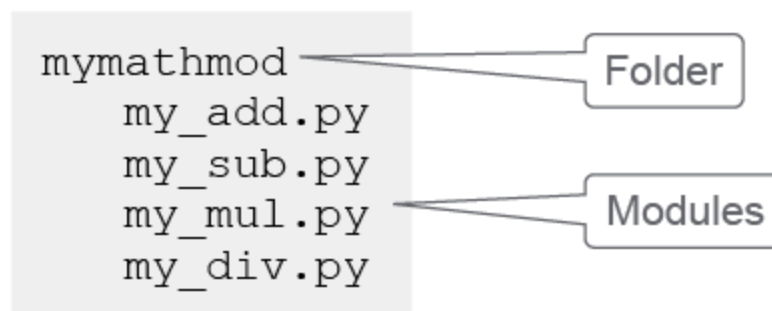
```
>>> import mymathmod as math
```

```
>>> math.my_div(100, 5)
```

Reusable Code with Packages

If you have more than a few modules, it might become difficult to keep track of them, and, therefore, packages become useful. A package is a hierarchical structure of the module's namespace. In the same way that modules help avoid collisions between global variables, packages help prevent collisions between modules.

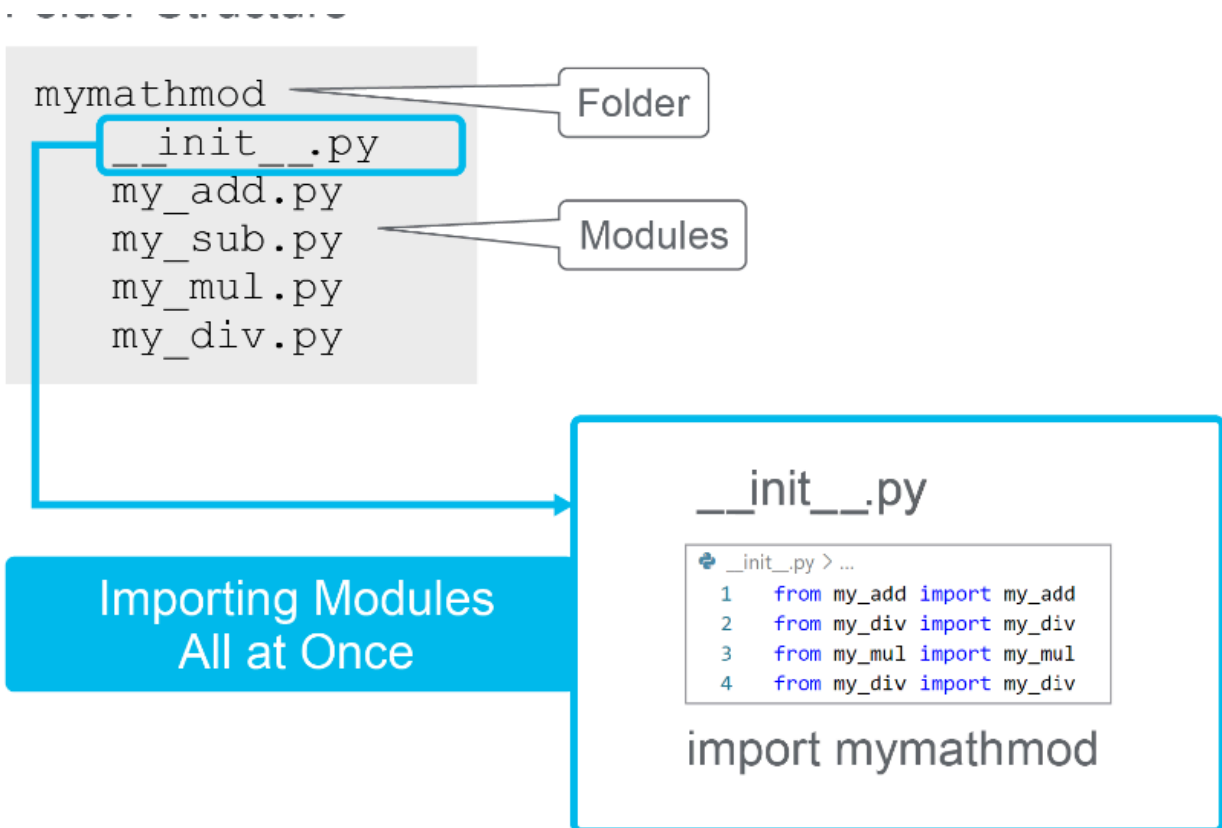
Folder Structure



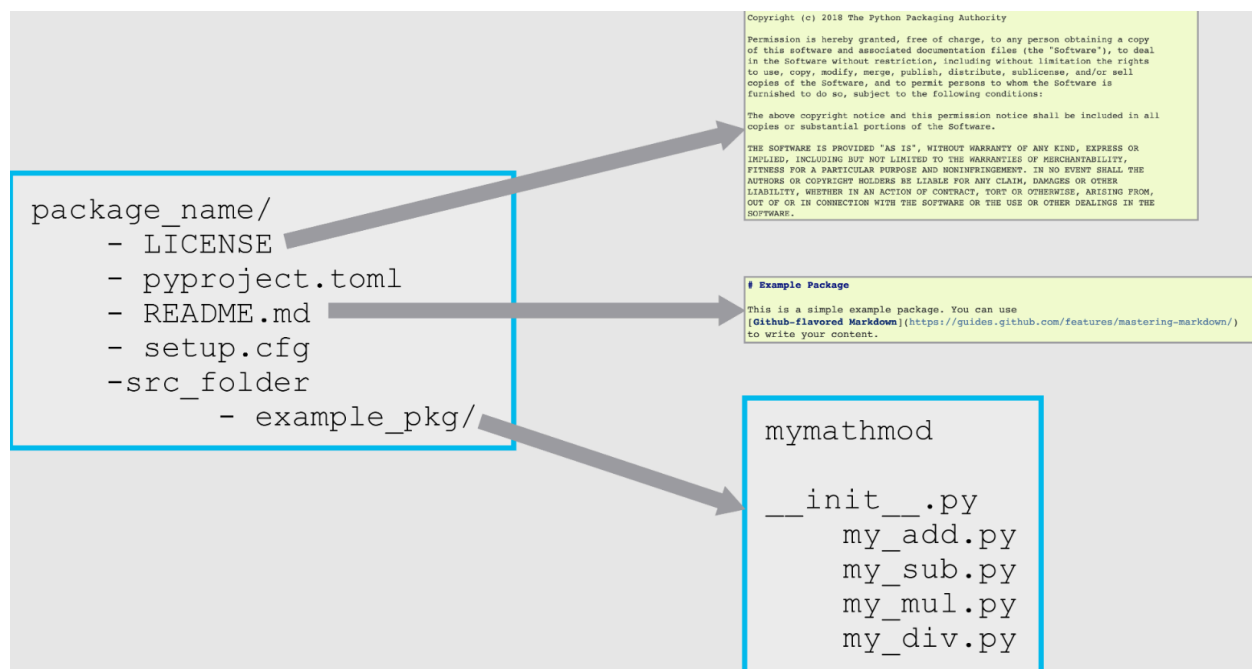
Importing Modules Individually

```
import mymathmod.my_add
import mymathmod.my_sub
import mymathmod.my_mul
import mymathmod.my_div
```

Notice that each module must be imported individually instead of using **import mymathmod** to import all of them.



If you would like to import all modules with the **import mymathmod** command, you must include a package initialization file (`__init__.py`). The `__init__.py` file in the example shows that the package initialization file is invoked when you import a package. Its execution allows all modules to be imported and can initialize code such as variables that can be used inside the package.



You should know the files that are or can be in an average package so you can understand the contents of packages you will use:

- A Python package's file structure should start with a folder with the package name. Inside the folder, include a LICENSE file, pyproject.toml file, README.md file, setup.cfg file, and a src_folder containing the package's source code.
- The LICENSE file tells users who install your package the terms under which they can use your package. To help you pick a license, you can go to <https://choosealicense.com>.
- The project.toml file is optional but recommended. It tells the build tools what is required for the build.
- README.md includes information such as the goal of the package, use cases, and quick start guides.
- The setup.cfg file includes the author, version, dependencies, description, and other information related to your package. Build a setup.cfg file and run it with the setup command.
- The src directory, which can have a different name if the developers prefer, should contain the file named `__init__.py` and the source code for the package.

- You can learn more on packages and packaging your own code from <https://packaging.python.org>.

Gathering and Validating User Input

Obtaining User Input

input() function , syntax is input(prompt)

Input Code

```
PRNE_Input.py > ...
1  import getpass
2
3  device_ip = input("Enter the device ip:")
4  username = input("Enter username:")
5  password = getpass.getpass("Password:")
6
7  print(f'Device ip: {device_ip}')
8  print(f'username: {username}')
9  print(f'password: {password}')
```

User Input Command

Output

```
C:\PRNE>python PRNE_Input.py
Enter the device ip:192.168.5.5
Enter username:eowen
Password:
Device ip: 192.168.5.5
username: eowen
password: password
```

Password is
Not Displayed

Since the value the user enters is always converted to a string, what do you do if you need an integer or a float value? The only way to get an integer or a float value is to convert the information. This conversion can be accomplished in multiple ways: you can

assign the user input to a variable and then convert or convert the value before it is assigned to the variable. Both ways are shown here:

```
value = input("Please enter an integer: ")
value = int(value)
```

or

```
value = int(input("Please enter an integer: "))
```

Note

When using the `int()` function, if the user's input cannot be converted to an integer, they will get a `ValueError`. For example, if the user enters `abc`, they will get a `ValueError` because `abc` cannot be converted to an integer.

When the script is run, you will notice in the example that the information that the user enters is displayed on the console. This behavior is fine usually except for passwords. For security reasons, you do not want to display passwords. Therefore, Python provides a built-in module called `getpass`. `getpass` allows users to enter a password without it displaying on the console. The code to use `getpass` follows:

```
password = getpass.getpass("Password:")
```

Basic Validation Check

Type Check	Checking the Data Type (str, int, float) <code>type(argument)</code> ➔ Date Type of Argument
Length Check	Checking the Length of a Sequence (str, list, dict, etc.) <code>len(sequence)</code> ➔ Numerical Value
Range Check	Checking if Value Falls within a Certain Range of Numbers <code>number in range(x, y)</code> ➔ True or False

If you accept user input, you need to make sure that the value entered is valid. There are three basic validation checks you can perform on user-entered data.

- Type check: Checks to see if the type of the value is correct. Even though the information entered from the input() function is converted to a string, you need to make sure that if you are looking for the user to enter a 1 that they don't enter 1.5 or the word "one."
- Length Check: Checks the length of the string entered. This check is useful when verifying the length of a password.
- Range Check: Checks whether the value entered falls within a certain range of numbers. Maybe, for example, the value must be between 0 – 100.

Password Length Check

```
password_is_too_short = True

while password_is_too_short:
    password = input('Enter password at least 5 characters: ')
    if len(password) >= 5:
        password_is_too_short = False
    else:
        print('Password entered is too short')

print(f'Your password entered is: {password}')
```

The example demonstrates how to check the length of a password. For example, let's propose that the requirement is that the password must be greater than or equal to five

characters. This example uses what's known as a flag check where the variable `password_is_too_short` is set to `True`. As long as `password_is_too_short` is `True`, the password doesn't meet the length requirement.

In the first line, the variable `password_is_too_short` is set to `True`. That variable will be used in the while loop to determine if it is executed. The while statement while `password_is_too_short` will run as long as the variable `password_is_too_short` is `True`.

The user is prompted for a password input and then an if statement checks the length of the password. If the length of the password entered is greater than or equal to 5, `password_is_too_short` will be set to `False` and the while loop will be exited. If it is less than 5, the else statement is executed, and the user is told that the password is too short and given another opportunity to enter the password again.

Command-Line Arguments

Command-line arguments are passed to the Python program when it is called from the command line. An example of how arguments are specified is the following:

```
C:\> python pythonprogram.py arg1 arg2 arg3
```

To use this functionality, you must code it into your script. In this section, you will learn how to use the `sys` and `argparse` modules to code this functionality.

Code

```
PRNE_ARG.py > ...
1  import sys
2
3  print(f'Number of arguments: {len(sys.argv)}')
4  print(f'Argument List: {str(sys.argv)}')
5  arg_list = sys.argv[1:]
6  print(arg_list)
```

Returns the number of arguments plus the file name.

Returns the file name and all arguments entered.

Output

```
C:\PRNE>python PRNE_ARG.py Test1 Test2 Test3
Number of arguments: 4
Argument List: ['PRNE_ARG.py', 'Test1', 'Test2', 'Test3']
['Test1', 'Test2', 'Test3']
```

Number of Arguments, Includes File Name

List of Arguments Including File Name

The simplest way to handle arguments is with the sys module. The sys module has been a standard library since the early days of Python; it provides access to some variables used or maintained by the interpreter. To use the sys module, you need to import it using the command:

```
import sys
```

Once imported, the sys module implements the arguments in a simple list structure named sys.argv. Line 4 of the example shows a print statement:

```
print(f'Argument List: {sys.argv}')
```

The print statement prints the sys.argv list as a string with the following result:

```
Argument List: ['PRNE_ARG.py', 'Test1', 'Test2', 'Test3']
```

The result shows that the list contains the Python file name at index 0 and then the arguments follow.

To use one of these arguments in your code, you need to access it at its index number. To retrieve the argument Test1 in the example, you would use the command **arg1 = sys.argv[1]**. This argument assigns the object that is at index 1 in the list sys.argv to the variable arg1.

If you want a list that does not include the Python program name, you can use the command in line 5 of the code example; it retrieves the objects in the list sys.argv starting at index 1 and ending at the end of the list.

```
arg_list = sys.argv[1:]
```

When you print arg_list, your results will not include the file name at index 0.

```
['Test1', 'Test2', 'Test3']
```

A function you may see used with sys.argv is the length function. Using the length function provides the number of arguments received from the command-line command, including the filename (which is at index 0 of the list). In the example, line 3 shows the following code:

```
print(f'Number of arguments: {len(sys.argv)}')
```

This command calculates the length of the list with **len(sys.argv)** and then prints 'Number of arguments: 4'. Keep in mind that this length will include the Python program

name, so you will need to subtract 1 from the result to get the correct number of arguments.

While the sys module is easy to use, it does not help your users understand what arguments should be used. Any help or error checking will need to be coded by you.

Code Using Argparse

```
PRNE_ARGPARSE.py > ...
1  import argparse
2
3  text = 'This program takes input for router information'
4  parser = argparse.ArgumentParser(description=text)
5  parser.add_argument("-R", "--router", help="Enter Router Name")
6  parser.add_argument("-IP", help="Enter ip address")
7
8  router = parser.parse_args()
9
10 print(f'The router name is {router.router} with ip address {router.IP}')
```

Python has a module with functionality for the verification of required and optional arguments. This module also includes help that can be accompanied by a default message, a list of accepted arguments, and any information you would like to provide about those arguments to the user. The argparse module has been available since Python 3.2 and enhanced the optparse module introduced in Python 2.7. To use the argparse module, you first need to import it with the following command:

```
import argparse
```

Once the module is imported, you need to initialize the parser, add a help statement, and add the required and optional arguments. Let's analyze the code from the example.

Line 3 of the script is assigning a help statement to a variable called text. This help statement can be used to help users understand basic information about the expected input. The variable, text, will be used when you initialize the parser. The text in the example lets the user know that the script will take router information.

```
text = 'This program takes input for router information.'
```

In line 4 of the code example, you create the parser and add the help statement. The ArgumentParser will hold the information to parse the command-line arguments into data types.

```
parser = argparse.ArgumentParser(description=text)
```

Lines 5 and 6 add your required or optional arguments with the **add_argument()** method.

```
parser.add_argument("-R", "--router", help=" Enter router name")
```

```
parser.add_argument("-IP", help=" Enter ip address")
```

The options available in the **add_argument()** method are shown:

```
parser.add_argument(short_option, long_option, help, action)
```

- Short option: Short representation of the argument that follows.
- Long option: Long representation of the argument that follows.
- Help: Message that will be displayed to help the user understand the argument.
- Action: If action is set to "store_true," the argument is optional. Otherwise, it will be blank.

In the example, there are two arguments: one is a router that can be added to the command line with a -R or: router and the other is an IP address that can be added to the command line with an -IP. Both have help information, so the user will get information about each if they use the help option.

In line 8 of the code example, **parser.parse_args()**, which holds the inputted arguments, is assigned to the variable `router` and used in a print statement. The print statement will print the router name and IP address to the console.

To use the arguments saved in `router`, you will need to use the `<variable_name>.<argument_option>`. Notice that to access the router name you would use `router.router` and to access the IP address you would use `router.IP`.

Executing File to Access Help

```
C:\PRNE>python PRNE_ARGPARSE.py -h
usage: PRNE_ARGPARSE.py [-h] [-R ROUTER] [-IP IP]

This program takes input for router information

optional arguments:
  -h, --help            show this help message and exit
  -R ROUTER, --router ROUTER
                        Enter Router Name
  -IP IP                Enter ip address
```

Access to Built-In Help

Help information you enter in the script.

Executing File with -R and -IP Arguments

```
C:\PRNE>python PRNE_ARGPARSE.py -R Router1 -IP 192.168.5.5
The router name is Router1 with ip address 192.168.5.5
```

When you execute the Python program with the -h option, you get help. This help was built in for you. In the past, you would have had to build it in the sys module yourself. This help saves you a lot of time and helps your users better understand what is expected.

Lastly, the Python program can be executed using the -R Router1 and the -IP 192.168.5.5 arguments. The script will take these arguments and print them out. The result is:

"The router name is Router1 with ip address 192.168.5.5".

Analyzing Exceptions and Error Management

Errors and Exceptions

As you spend time programming, you will inevitably make errors. Understanding the types of errors you will see in Python and when they occur will help you understand where to start debugging and how to solve the error. In Python, there are two main types of errors: syntax and exceptions.

SyntaxError

```
1 import requests
2
3 x = 5
4 y = 6
5
6 print('This is the PRNE class)
```

```
C:\PRNE>python syntaxerror.py
File "C:\PRNE\syntaxerror.py", line 6
    print('This is the PRNE class)
                                   ^
SyntaxError: EOL while scanning string literal
```

Exception

```
1 x = 1
2 y = 0
3
4 z = x / y
5
6 print(x)
```

```
C:\PRNE>python exceptionerror.py
Traceback (most recent call last):
  File "C:\PRNE\exceptionerror.py", line 4, in <module>
    z = x / y
        ^
ZeroDivisionError: division by zero
```

Syntax errors occur when the parser identifies an incorrect structure or syntax in Python. Syntax errors occur due to issues such as a misspelled keyword or a missing colon (:) at the end of a function. In the example, a (') is missing at the end of the string "This is the PRNE class" in the print statement, causing a syntax error.

Now that you understand what a syntax error is, let's look at how to read them. In syntax error section of the example, the first line of the error is:

File "C:\PRNE\syntaxerror.py", line 6

The first part of the line is the file where the error was found (syntaxerror.py) and the line the error was found on (line 6). The second line shows the code that the error was found in with a caret (^) showing where the error was discovered. Know that the caret (^) doesn't always point to the exact error location, but it should be close. Here the error was with the **print()** statement and the caret points to the end of the line. The last line explains this is a SyntaxError: EOL while scanning string literal.

Many code editors will highlight syntax errors in your code before you run it.

Exceptions are errors that occur during runtime. In other words, the program has the correct syntax, but something happened that the program couldn't handle. Note the exception error shown in the green box in the example. In the example, x is divided by y. The problem is that y = 0 and you can't divide by zero. This error stops the program and returns a ZeroDivisionError.

To read an exception error, it is best to start at the bottom of the error and read up. The error will list all the modules involved in the error with the most recent call last. Looking at the error in the example Exception block, the last line lists the exception and the exception object (information about the exception). The exception `ZeroDivisionError` and the exception object is “division by zero.” The next line up shows the code where the error occurred. The last line in this example gives you the file and the line in the file the error occurred in. There are several built-in exceptions that you may encounter.

Common Exceptions			
AttributeError	IOError	ImportError	IndexError
KeyError	KeyboardInterrupt	NameError	OverflowError
OSError	TypeError	ValueError	ZeroDivisionError

A few of the most common exceptions include:

- `AttributeError`: An attribute reference assignment fails.
- `IOError`: A system-related error, including I/O failures such as “file not found” or “disk full.”
- `ImportError`: An import statement fails to load a module.
- `IndexError`: A sequence subscript is out of range.
- `KeyError`: A mapping (dictionary) key is not found in the set of existing keys.
- `KeyboardInterrupt`: A user hit the interrupt key (normally Control-C or Delete).
- `NameError`: A local or global name is not found.
- `OverflowError`: The result of an arithmetic operation is too large to be represented.
- `OSError`: A function returns a system-related error.
- `TypeError`: An operation or function is applied to an object of inappropriate type.

- **ValueError:** A built-in operation or function receives an argument that has the right type but an inappropriate value, and the situation is not described by a more precise exception such as **IndexError**.
- **ZeroDivisionError:** The second argument of a division or modulo operation is zero.

Error Types

Syntax Errors

```

1  def hostname(name)
2  |    print(f'{name}')

```

C:\PRNE>python mis2.py
File "C:\PRNE\mis2.py", line 1
def hostname(name)
 ^
SyntaxError: invalid syntax

Missing
:

Runtime Errors

```

1  dev = {"name": "Router1", "ip": "192.168.5.1"}
2
3  print(dev["Hostname"])

```

C:\PRNE>python mis.py
Traceback (most recent call last):
File "C:\PRNE\mis.py", line 3, in <module>
print(dev["Hostname"])
KeyError: 'Hostname'

Hostname is not
defined in dictionary
dev

Syntax Errors

These errors are the most common and the easiest to correct. You will typically get an error message that starts with **SyntaxError** followed by a message. The error message normally gives the location in the code where Python noticed the issue, but other than that, it might not be beneficial. The prior example shows a syntax error where the function statement is missing the colon (:) at the end. There are two ways you could notice a syntax error. The first way is inside the Editor you are using. For example, the editor used earlier is VS Code, which indicates the location of an error with a red squiggly line. The second way is receiving a **SyntaxError** from the interpreter when you run the code. The error code gives you information about the line the error was discovered in and a caret to indicate where in the line the error was detected. Some common syntax errors follow.

- **Inconsistent Indentation:** The example shows inconsistent indentation. If you look at the two print statements in the function dev you will notice that one is indented four spaces and the other is indented two spaces. This will cause an “IndentationError: unindent does not match any outer indentation level.”

```
def dev(name, ip):  
    print(f'Router hostname {name}')
```



```
    print(f'Router ip {ip}')
```

Error

Unclosed Brackets: Unclosed brackets happen when you are missing one of the two brackets needed for a command. In the example, the first print statement is missing the closing parenthesis. This will give you the following error: “SyntaxError: unexpected EOF while parsing.”

```
def dev(name, ip):  
    print(f'Router hostname {name}')
```



```
    print(f'Router ip {ip}')
```

Error

Missing Quotation Marks: Missing quotation marks occur when you open a single or double quote and omit a closing mark or you omit an opening single or double quotation but use one for closing. This error will give you the following error: “SyntaxError: EOL while scanning string literal.”

```
def dev(name, ip):  
    print(f'Router hostname {name}')
```



```
    print(f'Router ip {ip})
```

Error

Python Keyword used as variable: Python keywords are reserved and cannot be used as variables. If you try to use a keyword, you will get the following error: “SyntaxError: invalid syntax” with a caret point close to the keyword being used.

```
if = gi0/1
def dev(name, ip, if):
    print(f'Router hostname {name}')
    print(f'Router interface {if}')
    print(f'Router ip {ip}')
```

Error

Runtime Errors

If the program is syntactically correct, but fails when you run it, there is a runtime error. Runtime errors include exceptions such as divide by zero or passing the wrong number of arguments to a function or method. Runtime errors also occur when a program gets caught in an infinite loop. In the example, the runtime error is a `KeyError`. The program has no syntax errors, but when you run it, the interpreter recognizes that you are trying to call a key in the dictionary `dev` that doesn't exist.

Semantic Errors

These errors are typically the most difficult to debug because there are no error messages. The code runs, and may produce an output, but the output is not correct. To troubleshoot these errors, it is important to understand what an output should be, based on specific inputs.

Semantic Errors

```
1  import netmiko
2
3  ✓ conn = netmiko.ConnectHandler(
4      '10.10.10.1',
5      username='cisco',
6      password='cisco',
7      device_type='cisco_ios'
8  )
9
10 run_cfg = conn.send_command('sh runnin-config')
```

This command has a typo, but Python could not know that.

In the example, you connect to a router running Cisco IOS, then send a command. The router should respond with the router output from that command. There is a typo in the sent command, so, instead of the expected output, you will get an error output from the router. This will not raise any Python errors or exceptions, but the code will not behave

as you expect. To correct this issue, you need to inspect the variable `run_cfg` and see that there is an issue with the command. This is a relatively easy error to locate and correct, but semantic errors can be much more difficult to find and debug.

Managing Exceptions

Managing exceptions allows you to handle errors and perform a meaningful action, such as display a message to a user or log a message. To handle exceptions, you can use the “try-except” block or the “try-except-finally” block.

```
try-except-else:
    try:
        Script to try
    except <ExceptionType>:
        Code to run if specific exception is raised
    else:
        Code to run if no exceptions are raised

try-except-else-finally:
    try:
        Script to try
    except <ExceptionType> as e:
        Code to run if specific exception is raised
        The variable e contains the error message for this exception
    else:
        Code to run if no exceptions are raised
    finally:
        Code that will always run at the end of exception handling.
```

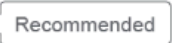
As shown in the “try-except-else” example, you write your script that you want to run in the try section. If the script executes properly, it skips the except section and continues to run the code in the else section. If there is an exception, it checks each except statement, and if one matches, it executes the code below it. You can have one or more except sections in the “try-except-else” block.

In the “try-except-else-finally” example, the try-except-else section works the same as in the “try-except-else” block example. The “finally” section is always run, whether the try is successful or not, and is used for code cleanup such as closing a file before the code exits. This code will run even if you issue the command `sys.exit()` in one of the except sections or the else section.

```
1 deviceip = {"R1": "192.168.5.5", "R2": "192.168.20.2", "R3": "192.168.50.2"}
2 try:
3     deviceip["R4"]
4 except:
5     print("Router doesn't exist.")
```



```
1 deviceip = {"R1": "192.168.5.5", "R2": "192.168.20.2", "R3": "192.168.50.2"}
2 try:
3     deviceip["R4"]
4 except KeyError:
5     print("Router doesn't exist.")
```



In the next example of a “try-except” block, there is a dictionary named deviceip. If a router is not present in the dictionary, the code displays a message to the user that states, "Router doesn't exist."

Using except with no ExceptionType is not recommended because it catches all exceptions, not a specific exception. In Python, this is known as a bare except. Using except by itself is acceptable after performing specific checks as a catch-all at the end, but it should not be used by itself.

The recommended way to perform exception handling is to check for specific ExceptionTypes first. The example checks for a KeyError. A KeyError is raised when a key is not found in a dictionary and it is accomplished with the following command:

except KeyError:

The code below the KeyError except statement will only execute if the exception is of the type KeyError.

```

1    x = 1
2    y = 0
3    try:
4        z = x / y
→ 5    except ZeroDivisionError as e:
6        print(e)

```

```

C:\PRNE>python exception1.py
division by zero

```

Variable holding
exception object

exception objects are the information that follows the ExceptionType and explains the error. If you would like to keep the exception object to print or send to a log file, you can use the following command:

```
except <ExceptionType> as <variable>:
```

The "as <variable>" will take the exception object and save it in the variable that you define. In the example, in line five you are holding the exception object for ZeroDivisionError in the variable e.

```

1    import sys
2    from netmiko import ConnectHandler
3    from netmiko import NetmikoAuthenticationException as AuthFailure
4
5    try:
6        conn = ConnectHandler('10.254.0.1', username='disco',
7                               password='cisco', device_type='cisco_ios'
8                               )
9    except AuthFailure as e:
10        output = e
11        sys.exit()
12    else:
13        output = conn.send_command('show running-config')
14    finally:
15        print(output)

```

The finally section executes whether the try statement is successful or not. In other words, it will always execute. To use a finally statement effectively, you need to logically plan your exception handling block to have something you always want to happen when the block is done running. This is not always necessary but can be a useful tool. In the example, you import the NetmikoAuthenticationException as AuthFailure. If AuthFailure is raised when you run the code under the try statement, you define the variable output as the raised error message, then issue `sys.exit()`. If the code under the try statement runs without error, you will define output as the router output from the command show running-config. Finally, even though you issued `sys.exit()` in the code under the except statement, the script will print the value of output, which will either be an error message or the router output. Instead of printing this information to the terminal, you can write it to a log file, update a database, or use the value of output in another logical way.

As you are building your programs, there may be times when you need to write your own custom exceptions.

```
1  import requests
2
3  ip = 'https://10.254.0.1:443/'
4  dn = '/restconf/data/Cisco-IOS-XE-native:native/interface/'
5  auth = ('cisco', 'cisco')
6  dash = '-' * 80
7
8  try:
9      response = requests.get(ip + dn, headers=headers, auth=auth, verify=False)
10     response.raise_for_status()
11 except requests.HTTPError as e:
12     filename = 'errors.log'
13     data = e
14 else:
15     filename = 'success.log'
16     data = response.content
17 finally:
18     with open(filename, 'a') as f:
19         print(data, dash, sep='\n', file=f)
```



Raise HTTPException if status code is not 200.

Catch HTTPException

In the example, you use the requests package to make an HTTP request to a connected services router (CSR) via Representational State Transfer Configuration Protocol (RESTCONF). The requests package has become the standard for making HTTP requests in Python. It takes the complexities that you would normally deal with when sending HTTP requests and makes them easy to code. To learn more about the Request library visit <https://pypi.org/project/requests/2.7.0/>.

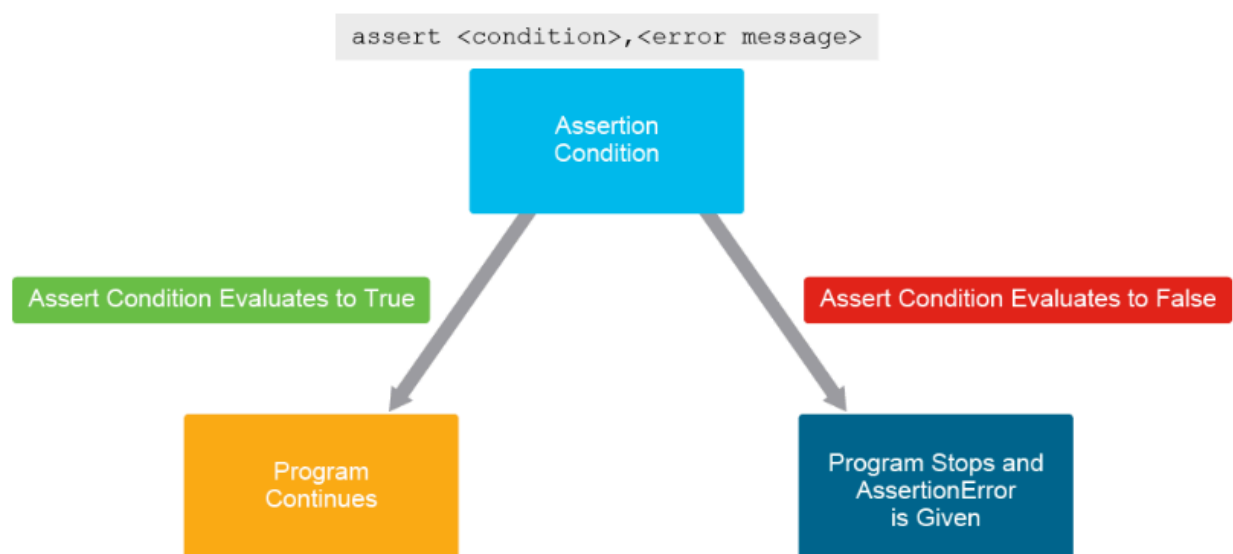
A successful request will have the status code 200. After you make a request with the request module, instead of automatically raising an exception if the request failed, the response object will have a status code that is not 200. The different status codes relate to different issues with the request. To raise an exception, you must use the response object's method `raise_for_status()` as in the bounding box on line 10 in the code example and the following code.

```
response.raise_for_status()
```

After attempting to make a request, the script will take one of two paths. If an exception was raised, the value of `filename` is set to `'errors.log'` and the value of `data` is set to the exception message. If no exception was raised, the value of `filename` will be `'success.log'` and the value of `data` will be the contents (or body) of the response containing XML data describing the interfaces on this device. In the `finally` block, regardless the values of `filename` and `data`, you write the value of `data` to the file specified by `filename`. In this example, the code in `finally` does the same thing every time, writes data to a file, but depending on how the code ran, you will either write an error message to an error log or write an output to a general log.

Assertions

Assertions are statements in your program used for debugging unrecoverable errors. The `assert` statement is evaluated to see if the condition is true or false. Unlike exceptions which display errors during code execution, assertions are used as a debugging aid.



Python has a built-in assert statement to test a condition and you place that assert statement in the area of the code you want to test. The assert statement is as follows:

```
assert <condition>,<error message>
```

The condition is a Boolean expression that is tested. If the condition is true, the program continues, but if the condition is false, assert stops the program and returns an AssertionError.

Code

```
1  import requests
2
3  ip = 'https://10.254.0.1:443'
4  dn = '/restconf/data/Cisco-IOS-XE-native:native/interface/'
5  auth = ('cisco', 'cisc')
6
7  response = requests.get(ip + dn, auth=auth, verify=False)
8
9  assertion_message = f'{response.status_code} {response.reason}'
10 assert response.status_code == 200, assertion_message
11 print(response.content)
```

Output

```
Traceback (most recent call last):
  File "requests_assertion.py", line 11, in <module>
    assert response.status_code == 200, assertion_message
AssertionError: 401 Unauthorized
```

he code shown in the example makes an HTTP request to a CSR router via RESTCONF. The assert statement condition states that the status code of the response must be 200, meaning successful, otherwise an AssertionError will be raised. In the example, the authorization has a typo in the password. The output message for the AssertionError is an f-string that consists of the actual status code of the response and the reason or description of that status code. You can see in the output that the AssertionError has been raised and the output explains that there was a 401 status code associated with an authorization failure or "Unauthorized." If the assertion test does not fail, you will print the XML contents of the request response to the terminal.

Crash Course Partial Notes

General

Check Class

type command

```
a = 10  
print(type(a))
```

Change Class Type

use str, int, etc.

```
Number_of_Nodes = 15  
print('our network is made of ' + str(Number_of_Nodes))
```

Formatting

```
f" {} {} "
```

variables inside the format all becomes strings and don't need to concatenate with +

```
Router_type = "ASR9K"  
SW_version = "6.4.2"  
Number_of_Nodes = 15
```

```
My_Network = f"Our network is made of {Number_of_Nodes} {Router_type} running S/W image  
{SW_version}"  
print(My_Network)
```

Built-in Method

predefined functions in Python with no need for imported modules

```
dir()
```

gives options for appending to the variable

```
print(dir(IP))
New_IP = IP.replace()
New_IP = IP.replace("172", "192")
```

Working with Files

#WORKING WITH FILES

```
import os
#print(os.getcwd())

#os.mkdir("TEST_LAB1")
#os.chdir("TEST_LAB1")
#print(os.getcwd())

print(os.listdir())
print(os.system("more Cisco1.txt"))
```

os.system command used “cat” but didn’t work with windows. “more” allows seeing contents of file on windows

IP Manipulation

replace octets

```
IP = "172.16.10.1"

New_IP = IP.replace("172", "192")
print(New_IP)
```

Split octets

```
IP = "172.16.10.1"
```

```
New_IP = IP.split(".")  
print(New_IP)
```

output:

```
['172', '16', '10', '1']
```

Append to IP

```
IP = "172.16.10.1"  
New_IP = IP.split(".")
```

```
New_IP.append("255.255.255.0")  
print(New_IP)
```

output: ['172', '16', '10', '1', '255.255.255.0']

Linux

CTRL-D ('control DROP) = exit()

nano easy text editor

to use Python3 interpreter for this file:

```
#!/usr/bin/env python3
```

this must be on first line of file

A recommended text editor on Linux is Geany

Definitions

for repetitive tasks

definition (has these fields)

print (f" strings + these fields ")

definition ("field1 =", "field 2 =")

example:

```
def ospf_process (process_id, network, mask, area):  
    print(f"router ospf {} \n network {network} {mask} {area}")
```

```
ospf_process ("10", "10.1.1.0", "255.255.255.0", "50")
```

